



المدرسة الوطنية للعلوم التطبيقية - آسفي
Ecole Nationale des Sciences Appliquées - SAFI
National School of Applied Sciences - SAFI

N° d'ordre :/.....

UNIVERSITÉ CADI AYYAD
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE SAFI

Filière : Ingénierie des Données et Intelligence Artificielle

Mémoire de Projet de Fin d'Études

Pour l'obtention du diplôme d'Ingénieur d'État en Informatique

Conception et Déploiement d'une Cloud Data Platform

Architecture Lakehouse sur AWS EKS

Rédigé et soutenu par :
Mohammed Dahiry

PARTENAIRES DE STAGE



Organisme d'accueil



Mission client

Sous la direction de :

Pr. EZZAHAR Jamal

Encadrant académique – ENSA Safi

M. EL GHALLAOUI Zine Labidine

Encadrant professionnel – SEVENAPP

Membres du Jury :

Pr. EZZAHAR Jamal

Pr. MADIIFI Mohammed

Pr. HARZALLA Driss

Pr. BENDAIDA Fatiha

Stage effectué chez **SEVENAPP** – Mission client : **EQDOM**

Période : Mars 2026 au Juillet 2026 • Soutenance : 26 Juin 2026

ANNÉE UNIVERSITAIRE 2025–2026

Dédicace

Dédicace

À mes très chers parents.

À ma mère, qui a œuvré pour ma réussite par son amour, son soutien, tous les sacrifices consentis et ses précieux conseils. Pour toute son assistance et sa présence dans ma vie, qu'elle reçoive à travers ce travail, aussi modeste soit-il, l'expression de mes sentiments les plus sincères et de mon éternelle gratitude.

À mon père, qui peut être fier et trouver ici le résultat de longues années de sacrifices et de privations pour m'aider à avancer dans la vie. Puisse Dieu faire en sorte que ce travail porte ses fruits. Merci pour les valeurs nobles, l'éducation et le soutien permanent que tu m'as apportés.

À mes frères, Salim et Anass, je dédie ce travail avec tous mes vœux de bonheur, de santé et de réussite. Vous m'avez soutenu depuis le début, et je vous en remercie pleinement.

À mes grands-pères, mes grands-mères, mes tantes et mon oncle, je ne vous remercierai jamais assez pour l'aide, le soutien et les prières que vous m'avez apportés.

À mes collègues de la promotion 2025/2026 du cycle d'ingénieur à l'ENSA.

À tous mes amis, pour votre soutien, votre présence et vos précieux encouragements.

Au Pr. EZZAHAR Jamal, mon encadrant académique, ainsi qu'à tous mes enseignants, je vous remercie d'avoir partagé avec moi votre passion pour l'enseignement. J'ai grandement apprécié votre soutien, votre implication et votre expérience tout au long de mon cursus.

À toute personne qui a cru en moi.

Je dédie ce travail.

Mohammed Dahiry

Remerciements

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce Projet de Fin d'Études.

Je remercie sincèrement mon encadrant académique, **Pr. EZZAHAR Jamal**, pour son suivi, sa disponibilité, ses orientations méthodologiques et ses précieux conseils tout au long de ce travail.

J'adresse également mes remerciements les plus sincères à **M. EL GHALLAOUI Zine Labidine** ainsi qu'à l'équipe **SEVENAPP**, pour leur accueil, leur accompagnement et la confiance accordée durant cette mission en tant que consultant Data auprès de **EQDOM**.

Je remercie les interlocuteurs chez **EQDOM** pour le contexte métier partagé, les échanges enrichissants et les éclairages apportés sur les enjeux data du secteur financier.

Que le corps professoral, le staff administratif et le secrétariat du département informatique de l'**ENSAS** trouvent ici mes vifs remerciements pour tout le travail effectué durant mes années de formation d'ingénieur d'État.

Que les membres du jury trouvent ici l'expression de ma reconnaissance pour avoir accepté d'évaluer mon travail.

Enfin, que tous ceux qui ont contribué à l'accomplissement de ce travail trouvent ici l'expression de mes remerciements les plus sincères.

Mohammed Dahiry

Résumé

Ce rapport présente la conception et la mise en place progressive d'une **Cloud Data Platform** moderne, réalisée dans le cadre d'un stage chez **SEVENAPP** en mission de **consultant Data** chez **EQDOM**. Le projet répond à un besoin de centralisation, de fiabilisation et d'exploitation industrielle des données afin de soutenir les usages analytiques, décisionnels et futurs cas d'usage data-driven de l'entreprise.

La solution proposée repose sur une infrastructure cloud établie autour d'une architecture **Lakehouse** déployée sur **AWS EKS**. Cette infrastructure permet d'héberger les services nécessaires à l'ingestion, au stockage, au traitement, à la transformation et à l'exposition des données dans un environnement scalable, sécurisé et exploitable. Elle constitue un socle technique unifié pour remplacer les échanges dispersés par une chaîne de traitement gouvernée, traçable et orientée production.

Les sources de données prises en charge couvrent principalement les bases opérationnelles de l'écosystème métier, notamment la **base Informix CBS**, la **base ONE** et la **base SmartStream**, ainsi que des fichiers métiers de formats variés tels que **CSV**, **XLSX**, **XLS** et autres fichiers structurés. Ces données sont intégrées à travers le pipeline data défini dans les spécifications fonctionnelles et techniques du projet. Elles transitent progressivement par les différentes zones du **Data Lake**, selon une logique de structuration et de qualité adaptée aux besoins BI.

Le pipeline cible permet d'alimenter le Data Lake, de préparer les données pour les **jobs BI**, puis de les transformer et les modéliser à l'aide de **dbt** et **Apache Spark**. Les traitements visent à produire des jeux de données fiables, historisés, documentés et prêts à l'analyse. Une attention particulière est portée à la protection des informations sensibles : les données confidentielles sont hashées à l'aide d'algorithmes avancés afin de préserver leur confidentialité et d'éviter toute exposition non maîtrisée.

Le projet intègre également une dimension d'exploitation indispensable à une plateforme data professionnelle. Les volets **monitoring**, **logging** et **dashboarding** permettent de suivre l'état de santé de l'infrastructure, d'observer l'exécution des pipelines, de centraliser les journaux applicatifs et de fournir des tableaux de bord facilitant le pilotage opérationnel. Ainsi, la plateforme ne se limite pas au stockage des données : elle propose un environnement complet pour industrialiser les flux, sécuriser les traitements et soutenir les besoins analytiques d'EQDOM.

Mots-clés : Cloud Data Platform, Lakehouse, Data Lake, AWS EKS, pipeline data, Informix CBS, ONE, SmartStream, fichiers métiers, BI, dbt, Apache Spark, monitoring, logging, dashboarding, sécurité des données.

Abstract

This report presents the design and progressive implementation of a modern **Cloud Data Platform**, carried out during an internship at **SEVENAPP** as a **Data Consultant** on assignment at **EQDOM**. The project addresses the need to centralize, secure and industrialize data flows in order to support analytical, decision-making and future data-driven use cases within the organization.

The proposed solution is based on an established cloud infrastructure built around a **Lakehouse** architecture deployed on **AWS EKS**. This infrastructure hosts the services required for data ingestion, storage, processing, transformation and exposure in a scalable, secure and operationally manageable environment. It provides a unified technical foundation that replaces fragmented data exchanges with a governed, traceable and production-oriented processing chain.

The platform targets several business data sources, mainly operational databases from the enterprise ecosystem, including the **Informix CBS database**, the **ONE database** and the **SmartStream database**, as well as business files in various formats such as **CSV**, **XLSX**, **XLS** and other structured files. These data sources are integrated through the data pipeline defined in the project's functional and technical specifications. Data progressively flows through the different zones of the **Data Lake**, following a structured approach aligned with quality, traceability and BI requirements.

The target pipeline feeds the Data Lake, prepares data for **BI jobs**, and enables transformation and modeling using **dbt** and **Apache Spark**. The processing layer is designed to produce reliable, historical, documented and analysis-ready datasets. Particular attention is given to the protection of sensitive information : confidential fields are hashed using advanced algorithms to preserve data confidentiality and prevent uncontrolled exposure.

The project also includes essential operational capabilities required for a professional data platform. The **monitoring**, **logging** and **dashboarding** layers make it possible to supervise infrastructure health, observe pipeline execution, centralize application logs and provide dashboards for operational follow-up. As a result, the platform is not limited to data storage ; it delivers a complete environment for industrializing data flows, securing processing activities and supporting EQDOM's analytical needs.

Keywords : Cloud Data Platform, Lakehouse, Data Lake, AWS EKS, data pipeline, Informix CBS, ONE, SmartStream, business files, BI, dbt, Apache Spark, monitoring, logging, dashboarding, data security.

ملخص

يقدم هذا التقرير تصميم وتنفيذ منصة بيانات سحابية حديثة، تم إنجازها في إطار تدريب لدى شركة SEVENAPP ضمن مهمة استشارية في مجال البيانات لفائدة EQDOM. يهدف المشروع إلى توحيد مصادر البيانات، تحسين موثوقية تدفقاتها، وتوفير أساس منظم يدعم التحليل واتخاذ القرار وحالات الاستعمال المستقبلية المعتمدة على البيانات. تعتمد المنصة المقترحة على بنية تحتية سحابية قائمة على معمارية Lakehouse منشورة فوق AWS EKS. وتوفر هذه البنية بيئة قابلة للتوسع وأمنة لاستيعاب البيانات، تخزينها، معالجتها، تحويلها، ثم إتاحتها للاستعمالات التحليلية والمهنية. كما تشكل هذه المنصة قاعدة تقنية موحدة تسمح بتعويض التبادلات المتفرقة بسلسلة معالجة منظمة، قابلة للتتبع وموجهة نحو الاستغلال الإنتاجي.

تشمل مصادر البيانات المستهدفة قواعد تشغيلية أساسية مثل قاعدة Informix CBS ، قاعدة ONE ، وقاعدة SmartStream ، إضافة إلى ملفات مهنية بصيغ متعددة مثل CSV و XLSX و XLS وغيرها من الملفات الهيكلية. تمر هذه البيانات عبر خط معالجة محدد في المواصفات الوظيفية والتقنية للمشروع، ثم يتم إدخالها تدريجياً إلى مناطق Data Lake وفق منطق يراعي الجودة، التتبع، ومتطلبات ذكاء الأعمال.

يسمح خط المعالجة المستهدف بتغذية Data Lake ، تحضير البيانات لأعمال BI ، ثم تحويلها ونمذجتها باستعمال dbt و Apache Spark . وتهدف هذه المعالجات إلى إنتاج بيانات موثوقة، مؤرخة، موثقة، وجاهزة للتحليل. كما يتم إيلاء أهمية خاصة لحماية المعطيات الحساسة، حيث يتم تجزئة الحقول السرية باستعمال خوارزميات متقدمة من أجل الحفاظ على سريتها ومنع أي تعرض غير مضبوط لها.

يتضمن المشروع كذلك جوانب تشغيلية أساسية لمنصة بيانات احترافية، من خلال logging و monitoring و dashboarding . وتسمح هذه المكونات بمتابعة صحة البنية التحتية، مراقبة تنفيذ خطوط المعالجة، تجميع السجلات التطبيقية، وتوفير لوحات قيادة تساعد على التتبع التشغيلي. وبذلك لا تقتصر المنصة على تخزين البيانات فقط، بل تقدم بيئة متكاملة لتصنيع تدفقات البيانات، تأمين المعالجات، ودعم الاحتياجات التحليلية لدى EQDOM .

الكلمات المفتاحية - منصة بيانات سحابية، Lakehouse ، Data Lake ، AWS EKS ، خط معالجة البيانات، Informix ، ONE ، SmartStream ، ملفات مهنية، BI ، dbt ، Apache Spark ، logging ، monitoring ، dashboarding ، أمن البيانات.

Table des matières

Dédicace

Remerciements

Résumé

Abstract

Résumé en arabe

Liste des abréviations

Introduction générale 1

1 CONTEXTE GÉNÉRAL DU PROJET 3

1.1 Organisme d'accueil : SEVENAPP 3

1.1.1 Fiche signalétique 3

1.1.2 Domaines d'intervention 4

1.1.3 Références clients 5

1.1.4 Organigramme simplifié 6

1.2 Contexte client : EQDOM 7

1.2.1 Présentation du client 7

1.2.2 Fiche signalétique 7

1.2.3 Aperçu historique 8

1.2.4 Organigramme simplifié 8

1.2.5 Réseau d'agences 9

1.2.6 Enjeux data du contexte financier 9

1.2.7 Motivation de la mission 10

1.3 Cadre du projet 10

1.4 Problématique 11

1.5 Méthodologie de conduite 12

1.6 Planification du projet 13

1.7 Apports attendus 14

Conclusion du chapitre 1 14

2 ÉTAT DE L'ART 15

2.1 Évolution des plateformes de données 16

2.2	Architecture Lakehouse et modèle médaille	16
2.3	Kubernetes et approche cloud-native	17
2.4	Pattern Operator	18
2.5	Ingestion batch et streaming	19
2.6	Transformation distribuée et orchestration	19
2.7	Virtualisation SQL et Business Intelligence	20
2.8	Infrastructure as Code et CI/CD	21
2.9	Observabilité	21
	Conclusion du chapitre 2	22
3	ANALYSE ET SPÉCIFICATION DES BESOINS	23
3.1	Problématique	23
3.2	Objectifs du projet	23
3.3	Périmètre fonctionnel	24
3.4	Exigences non fonctionnelles	25
3.5	Acteurs et responsabilités	25
3.5.1	Diagrammes de compréhension fonctionnelle et technique	26
3.6	Scénario de référence du pipeline data	31
3.7	Contraintes de réalisation	31
	Conclusion du chapitre 3	32
4	CONCEPTION DE LA SOLUTION	33
4.1	Architecture globale	33
4.2	Déploiement physique sur AWS EKS	34
4.2.1	Landing zone	34
4.2.2	Node groups	35
4.3	Stratégie des environnements DEV, UAT et PROD	36
4.4	Stratégie des namespaces Kubernetes	37
4.5	Conception du pipeline médaille	37
4.6	Sécurité et gouvernance technique	38
4.7	Observabilité, logging et dashboarding	38
4.8	Organisation technique des livrables	39
	Conclusion du chapitre 4	39
5	RÉALISATION ET MISE EN ŒUVRE	41
5.1	Stack technologique implémentée	41
5.2	État d'avancement par phase	43
5.3	Phase 1 — Landing zone et socle Kubernetes	44
5.4	Phase 2 — Stockage Lakehouse et métastore	51
5.5	Phase 3 — Ingestion, Kafka et CDC	54
5.6	Phase 4 — Spark, dbt et Airflow	56
5.7	Phase 5 — Serving avec Dremio	61
5.8	Phase 6 — Exposition sécurisée Cloudflare	63
5.9	Phase 7 — Monitoring, logging et dashboarding	63
5.10	Automatisation, CI/CD et cycle de vie	64
5.11	Difficultés rencontrées et mitigations	66

Conclusion du chapitre 5	66
6 CONCLUSION ET PERSPECTIVES	68
6.1 Bilan du projet	68
6.2 Limites	68
6.3 Perspectives	68
6.4 Apports personnels	69
Bibliographie	70

Table des figures

1.1	Logo SEVENAPP	3
1.2	Domaines d'intervention de SEVENAPP	5
1.3	Exemples de références clients de SEVENAPP	6
1.4	Organigramme simplifié de SEVENAPP	7
1.5	Logo EQDOM	7
1.6	Organigramme simplifié d'EQDOM	9
1.7	Organisation métier autour de la plateforme data	11
1.8	Taxonomie de la problématique data du projet	12
1.9	Démarche agile appliquée au projet	12
1.10	Diagramme de Gantt du projet	13
2.1	Architecture générale de la plateforme Data Lakehouse sur Kubernetes	15
2.2	Évolution des architectures de données vers le Lakehouse	16
2.3	Organisation du Lakehouse selon le modèle médaillon	17
2.4	Architecture infrastructure de la plateforme Data Lakehouse	17
2.5	Rôle de Kubernetes dans l'isolation et l'orchestration des composants	18
2.6	Principe de fonctionnement du pattern Operator	18
2.7	Modes d'ingestion batch et streaming vers la couche Bronze	19
2.8	Pipeline data end-to-end de la plateforme	20
2.9	Chaîne de transformation et d'orchestration des traitements	20
2.10	Exposition SQL et consommation BI des données Gold	21
2.11	Chaîne Infrastructure as Code et CI/CD	21
2.12	Piliers d'observabilité de la plateforme	22
3.1	Diagramme de flux de données de la plateforme	27
3.2	Diagramme d'activités du pipeline data	28
3.3	Diagramme d'infrastructure cloud cible	30
4.1	Architecture logique en couches de la Cloud Data Platform	33
4.2	Socle cible AWS/EKS retenu pour la conception	35
4.3	Organisation cible des environnements DEV, UAT et PROD	36
4.4	Flux de données médaillon cible	37
5.1	Principales technologies utilisées dans la Cloud Data Platform	42
5.2	Clés KMS gérées par le client pour le chiffrement de la plateforme	45
5.3	Validations terminal WSL du socle déployé	46
5.4	Cluster EKS, workloads et capacité compute après déploiement	47

5.5	Stockage persistant, sécurité réseau et VPC AWS réalisés	48
5.6	Associations de la première table de routage du VPC	49
5.7	Associations de la deuxième table de routage du VPC	49
5.8	Associations de la troisième table de routage du VPC	49
5.9	NAT Gateway utilisé pour la sortie contrôlée des sous-réseaux privés	50
5.10	Adresse IP élastique associée au NAT Gateway	51
5.11	Bases PostgreSQL utilisées par la source de démonstration et le Hive Me- tastore	52
5.12	Stockage objet MinIO et premières zones Lakehouse	53
5.13	Zone Gold matérialisée dans MinIO	54
5.14	Composants Kafka déployés et en fonctionnement dans Kubernetes	55
5.15	Flux NiFi réalisés pour l'ingestion des sources métier	56
5.16	Composants Spark déployés et en fonctionnement dans Kubernetes	57
5.17	Orchestration Airflow du pipeline médaillon	58
5.18	Traitements dbt, ingestion mensuelle et suivi Spark	59
5.19	Résultats de transformation Bronze, Silver et Gold	60
5.20	Indicateurs métier produits à partir de la couche Gold	61
5.21	Contenu, objectifs et enjeux de la couche BI	61
5.22	Dashboard BI de synthèse alimenté par la couche Gold	62
5.23	Dashboard BI d'analyse métier du portefeuille	62
5.24	Exposition sécurisée réalisée avec Cloudflare	63
5.25	Artefacts d'automatisation, projet dbt et image Spark	65
5.26	Identités AWS et backend R2 utilisés par l'automatisation	66

Liste des tableaux

1.1	Fiche signalétique de SEVENAPP	4
1.2	Fiche signalétique d'EQDOM	8
1.3	Répartition des agences EQDOM par ville	9
1.4	Planification synthétique du projet	13
2.1	Comparaison synthétique des architectures data	16
2.2	Operators mobilisés dans le projet	19
3.1	Sources de données retenues dans le périmètre	24
3.2	Exigences fonctionnelles principales	24
3.3	Exigences non fonctionnelles	25
3.4	Acteurs du système	26
3.5	Critères d'acceptation du scénario de référence	31
4.1	Rôle des environnements de la plateforme	36
4.2	Namespaces platform-* alignés avec le cahier des charges	37
4.3	Rôle des couches médaillon	38
4.4	Conception de l'observabilité	39
5.1	Stack technologique principale	41
5.2	Matrice d'avancement et livrables par phase	43
5.3	DAGs Airflow livrés	57
5.4	Réalisation de l'observabilité	64

Liste des abréviations

Sigle	Signification
ADR	Architecture Decision Record
API	Application Programming Interface
AWS	Amazon Web Services
BI	Business Intelligence
CBS	Core Banking System
CDC	Change Data Capture
CI/CD	Intégration / Déploiement continu
CMK	Customer Managed Key (clé KMS)
CNPG	CloudNativePG (opérateur PostgreSQL)
CRD	Custom Resource Definition
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph (Airflow)
DBT	Data Build Tool
Dremio	Plateforme SQL lakehouse pour l'exploration et la consommation des données
EBS	Elastic Block Store
ECR	Elastic Container Registry
EKS	Elastic Kubernetes Service
ELB	Elastic Load Balancer
ESN	Entreprise de Services du Numérique
ETL	Extract, Transform, Load
ELT	Extract, Load, Transform
Grafana	Plateforme de visualisation, de supervision et de dashboarding
HMS	Hive Metastore
IaC	Infrastructure as Code
IAM	Identity and Access Management
Informix CBS	Base CBS hébergée sur IBM Informix
IRSA	IAM Roles for Service Accounts
K8s	Kubernetes
KMS	Key Management Service
KPI	Key Performance Indicator

Sigle	Signification
Lakehouse	Architecture combinant les principes du Data Lake et du Data Warehouse
Fluent Bit	Agent léger de collecte, de filtrage et de transfert des logs
MinIO	Stockage objet compatible avec l'API S3
MVP	Minimum Viable Product
NAT	Network Address Translation
NiFi	Apache NiFi, outil d'ingestion, d'orchestration et de routage des flux de données
OIDC	OpenID Connect
OLTP	Online Transaction Processing
ONE	Source de données opérationnelle intégrée au pipeline data
OpenObserve	Plateforme d'observabilité pour la centralisation, la recherche et l'analyse des logs
PFE	Projet de Fin d'Études
Power BI	Outil de Business Intelligence et de restitution décisionnelle
Prometheus	Système de collecte de métriques et de supervision
PSA	Pod Security Admission
RBAC	Role-Based Access Control
R2	Cloudflare R2 (stockage objet)
S3	Simple Storage Service
SLA	Service Level Agreement
Spark	Apache Spark, moteur distribué de traitement et de transformation des données
SmartStream	Source de données opérationnelle intégrée au pipeline data
SoW	Statement of Work (cahier des charges)
SQL	Structured Query Language
XLS	Format de fichier tableur Microsoft Excel historique
XLSX	Format de fichier tableur Microsoft Excel moderne

Introduction générale

Dans un contexte où la donnée devient un actif stratégique, la capacité à la collecter, la fiabiliser, la transformer et l'exposer de manière gouvernée constitue un enjeu majeur pour les organisations financières.

Les organisations financières exploitent aujourd'hui des volumes de données de plus en plus importants, issus de systèmes transactionnels, d'applications métier, de fichiers opérationnels et d'outils analytiques. Cette richesse informationnelle devient cependant difficile à valoriser lorsque les données restent dispersées dans plusieurs silos, lorsque les traitements sont exécutés par des scripts isolés, ou lorsque l'absence d'observabilité rend les incidents difficiles à diagnostiquer. Dans ce contexte, la mise en place d'une plateforme de données industrialisée constitue un levier stratégique pour fiabiliser la collecte, structurer la transformation et accélérer l'accès aux indicateurs décisionnels.

Le présent Projet de Fin d'Études s'inscrit dans cette problématique. Réalisé dans le cadre d'un stage chez **SEVENAPP** [1], en mission de consultant Data auprès de **EQDOM** [2], il porte sur la conception et la mise en œuvre d'une **Cloud Data Platform** moderne, déployée sur **AWS EKS** [3], fondée sur une architecture **Lakehouse** [4] et structurée selon le modèle **médaille** [5] : Bronze, Silver et Gold.

L'objectif du projet n'est pas seulement d'assembler des outils open source. Il s'agit de construire un socle reproductible, sécurisé et exploitable, capable de couvrir l'ensemble du cycle de vie de la donnée : ingestion batch et streaming, stockage objet, catalogage, transformations distribuées, orchestration, exposition SQL/BI, supervision et automatisation de l'infrastructure. Le projet accorde également une attention particulière aux contraintes opérationnelles : gestion des secrets, isolation Kubernetes [6], optimisation des coûts, traçabilité des choix d'architecture et capacité à maintenir un environnement fiable et industrialisable.

La solution proposée repose sur un ensemble cohérent de technologies : Terraform [7] pour l'Infrastructure as Code, Cloudflare R2 [8] pour le backend d'état Terraform, Amazon EKS pour l'orchestration Kubernetes, MinIO [9] pour le stockage S3-compatible, Apache Iceberg [10] et Hive Metastore [11] pour la couche Lakehouse, Kafka [12]/Strimzi [13] et Debezium [14] pour la capture de changements, Apache Spark [15] et dbt [16] pour les transformations, Apache Airflow [17] pour l'orchestration, Dremio [18] pour la virtualisation SQL, puis Prometheus [19], Grafana [20], Fluent Bit [21] et OpenObserve [22] pour l'observabilité.

Ce rapport présente successivement le contexte professionnel et méthodologique du projet, l'état de l'art des architectures data modernes, l'analyse des besoins, la conception retenue, la réalisation technique de la plateforme, ainsi que les limites et perspectives d'évolution.

Les phases encore dépendantes d'un déploiement complet sur compte cloud sont intégrées comme trajectoire d'industrialisation, afin de fournir une vision complète et défendable de la plateforme cible.

Organisation du mémoire

- Chapitre 1 : présente l'organisme d'accueil, le contexte client et la méthodologie de travail ;
- Chapitre 2 : introduit les concepts et technologies nécessaires à la compréhension de la solution ;
- Chapitre 3 : formalise les besoins fonctionnels et non fonctionnels ;
- Chapitre 4 : décrit la conception globale et les choix d'architecture ;
- Chapitre 5 : détaille la réalisation technique et l'état d'avancement ;
- Chapitre 6 : conclut le travail et propose les perspectives d'amélioration.

Chapitre 1 : CONTEXTE GÉNÉRAL DU PROJET

Ce chapitre présente le cadre professionnel du Projet de Fin d'Études, l'organisme d'accueil, le contexte client et la démarche de conduite adoptée. Il permet de situer la contribution technique dans un environnement réel de transformation digitale et de data engineering.

1.1 Organisme d'accueil : SEVENAPP

SEVENAPP, opérant sous la marque **Seven**, est une entreprise de services numériques spécialisée dans l'accompagnement des organisations autour de la transformation digitale, de la data et de l'intelligence artificielle. Son positionnement consiste à intervenir comme partenaire technologique auprès d'acteurs publics et privés, en mettant à disposition des consultants capables de concevoir, industrialiser et documenter des solutions numériques. La figure 1.1 présente l'identité visuelle de l'organisme d'accueil.



FIGURE 1.1 – Logo SEVENAPP

Dans le cadre de ce stage, le rôle occupé est celui de **consultant Data**. La mission s'inscrit dans une logique d'ingénierie : comprendre un besoin de plateforme, proposer une architecture, produire une implémentation reproductible, documenter les choix et préparer une démonstration technique.

1.1.1 Fiche signalétique

Le tableau 1.1 synthétise les informations principales de SEVENAPP et précise son rôle dans le projet.

TABLE 1.1 – Fiche signalétique de SEVENAPP

Élément	Description
Dénomination	SEVENAPP, opérant sous la marque Seven
Nature	Entreprise de Services du Numérique orientée Digital, Data et Intelligence Artificielle
Date de création	2018
Siège	Casablanca, Maroc
Positionnement	Accompagnement des organisations dans la transformation digitale, l’industrialisation des produits data et l’intégration de solutions innovantes
Domaines d’expertise	Consulting, Digital, Data, Intelligence Artificielle, plateformes banking, Data Lakehouse et IA générative
Effectif public communiqué	Plus de 45 consultants et experts
Rôle dans le projet	Organisme d’accueil du stage et cadre de réalisation de la mission de consultant Data auprès d’EQDOM

Cette fiche met en évidence un positionnement orienté Digital, Data et Intelligence Artificielle, cohérent avec la mission de conception d’une Cloud Data Platform.

1.1.2 Domaines d’intervention

Les activités de l’organisme d’accueil peuvent être regroupées autour de trois axes :

- **transformation digitale** : modernisation des processus, intégration d’applications et accompagnement métier ;
- **data engineering** : pipelines, plateformes, gouvernance, ingestion, transformation et exposition des données ;
- **intelligence artificielle** : exploitation avancée des données, automatisation et cas d’usage analytiques.

La figure 1.2 illustre ces domaines d’intervention et leur complémentarité.

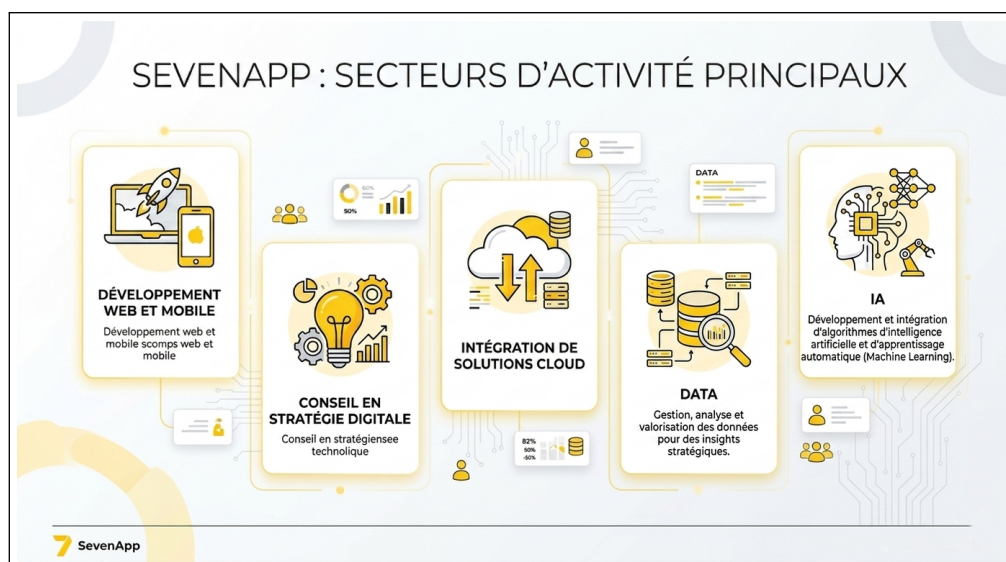


FIGURE 1.2 – Domaines d'intervention de SEVENAPP

1.1.3 Références clients

La diversité des références clients de SEVENAPP illustre son positionnement transversal auprès d'acteurs financiers, d'organismes publics et d'entreprises issues de secteurs variés. Ces missions couvrent notamment la refonte de parcours digitaux, la mise en place de solutions CRM, la structuration de dispositifs data, l'automatisation de processus et l'accompagnement à la gouvernance des données.

La figure 1.3 présente quelques références représentatives de cette diversité sectorielle.



FIGURE 1.3 – Exemples de références clients de SEVENAPP

1.1.4 Organigramme simplifié

La figure 1.4 situe la mission Cloud Data Platform dans une organisation simplifiée : elle relève du pôle Data & IA, lui-même rattaché aux activités digitales de SEVENAPP.

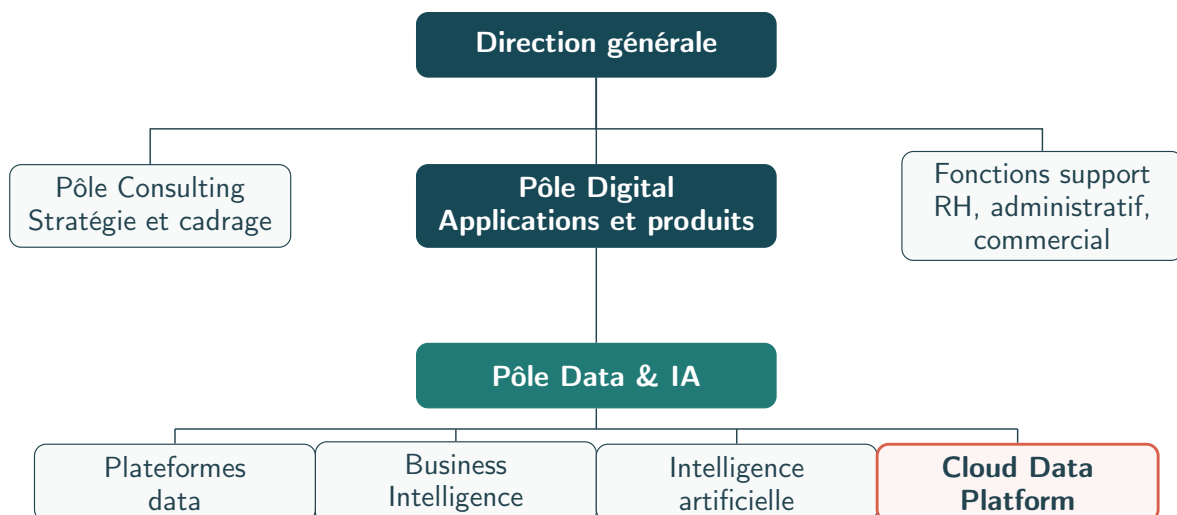


FIGURE 1.4 – Organigramme simplifié de SEVENAPP

1.2 Contexte client : EQDOM

1.2.1 Présentation du client

EQDOM est un acteur marocain du financement et du crédit à la consommation. Son activité repose sur la gestion de parcours clients, de dossiers de financement, de données opérationnelles et d'indicateurs de pilotage. Dans ce type d'environnement, la donnée joue un rôle central : elle alimente l'analyse du risque, le suivi commercial, la conformité, le reporting interne et les tableaux de bord décisionnels.

La figure 1.5 présente l'identité visuelle du client de la mission.



FIGURE 1.5 – Logo EQDOM

1.2.2 Fiche signalétique

Le tableau 1.2 résume le profil d'EQDOM, son activité et le contexte dans lequel s'inscrit la mission Data.

TABLE 1.2 – Fiche signalétique d’EQDOM

Élément	Description
Dénomination sociale	EQDOM
Forme juridique	Société anonyme à Conseil d’Administration
Date de création	Septembre 1974
Siège social	127, Boulevard Zerktouni, Casablanca
Secteur d’activité	Crédit à la consommation et financement automobile
Produits principaux	Prêts personnels, crédit automobile, financement d’équipement et solutions de crédit affecté
Réseau commercial	24 agences réparties dans les principales villes du Royaume, complétées par des intermédiaires agréés et des partenaires distributeurs
Contexte de la mission	Client final de la mission Data portée par SEVENAPP, avec un besoin de modernisation et de fiabilisation des flux de données

1.2.3 Aperçu historique

EQDOM est considérée comme l’une des premières sociétés marocaines spécialisées dans le financement et le crédit à la consommation. Créée en 1974 à l’initiative de la Société Nationale d’Investissement et de la Caisse de Dépôt et de Gestion, elle avait initialement pour vocation de faciliter l’acquisition de biens d’équipement par les ménages marocains et de soutenir le développement du secteur de l’électroménager.

L’entreprise a connu une première étape structurante en 1978 avec son introduction à la Bourse de Casablanca. À partir des années 1990, EQDOM a engagé une dynamique d’élargissement de son activité, marquée par la diversification de ses produits et l’extension progressive de son réseau d’agences. Cette évolution lui a permis de passer d’un positionnement centré sur le financement d’équipement à une offre plus large couvrant les prêts personnels, le crédit automobile et les solutions de financement affecté.

Dans les années 2000, l’entreprise a renforcé son ancrage dans le secteur financier marocain à travers son adossement à un groupe bancaire de référence. Plus récemment, l’évolution du paysage bancaire marocain et les opérations de réorganisation actionnariale ont inscrit EQDOM dans une nouvelle phase de transformation. Dans ce contexte, la donnée devient un levier essentiel pour améliorer le pilotage commercial, fiabiliser le reporting, suivre les risques et accompagner la digitalisation des parcours clients.

1.2.4 Organigramme simplifié

La figure 1.6 propose une lecture simplifiée de l’organisation d’EQDOM. Elle met en évidence le rattachement de la plateforme Lakehouse aux activités SI et Data qui soutiennent les opérations, le pilotage et le reporting.

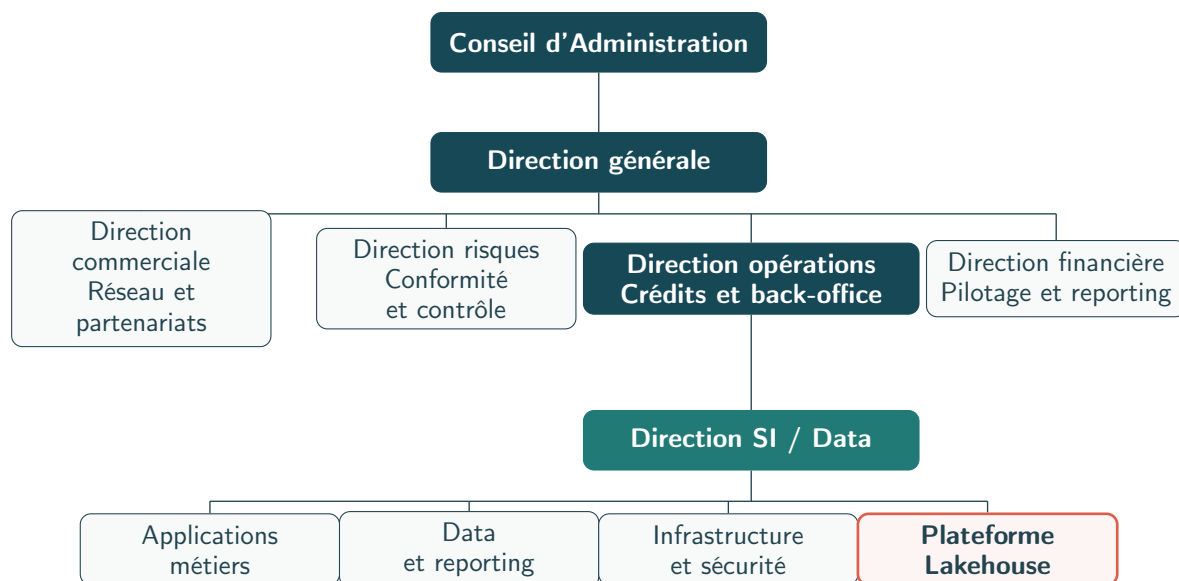


FIGURE 1.6 – Organigramme simplifié d'EQDOM

1.2.5 Réseau d'agences

EQDOM s'appuie sur un réseau de distribution directe afin d'assurer une présence de proximité auprès de ses clients, de ses partenaires automobiles et des organismes conventionnés. La documentation institutionnelle consultée indique un total de **24 agences** réparties dans les principales villes du Royaume.

Le tableau 1.3 détaille cette répartition. Il montre notamment le poids de Casablanca, qui concentre six agences, devant Rabat avec trois agences.

TABLE 1.3 – Répartition des agences EQDOM par ville

Ville	Agences	Ville	Agences
Casablanca	6	Rabat	3
Marrakech	2	Tanger	2
Settat	1	Fès	1
Salé	1	Oujda	1
Agadir	1	Safi	1
Tétouan	1	Meknès	1
El Jadida	1	Kénitra	1
Béni Mellal	1	Total	24

1.2.6 Enjeux data du contexte financier

Le contexte financier impose des exigences particulières :

- **fiabilité** : les traitements doivent produire des résultats cohérents et vérifiables ;
- **traçabilité** : les données doivent pouvoir être suivies depuis la source jusqu'à l'indicateur ;
- **sécurité** : les accès aux infrastructures et aux interfaces doivent être contrôlés ;

- **scalabilité** : la plateforme doit pouvoir absorber l'évolution des volumes et des usages ;
- **observabilité** : les erreurs, lenteurs et indisponibilités doivent être détectables rapidement.

1.2.7 Motivation de la mission

La mission consiste à proposer une base technique permettant de centraliser les flux de données et de les organiser sous forme de Lakehouse. Le projet n'a pas vocation à remplacer immédiatement tous les systèmes existants ; il fournit un socle reproductible et extensible sur lequel des cas d'usage métiers peuvent ensuite être branchés progressivement.

Le scénario de démonstration retenu s'appuie sur des données représentatives, notamment des tables clients et des données budgétaires d'agences. Ce choix permet d'illustrer la chaîne complète : ingestion depuis une source PostgreSQL, capture CDC, écriture Bronze, transformations Silver/Gold et exposition SQL.

Le secteur du crédit et du financement exige des systèmes d'information fiables, auditables et capables de produire rapidement des indicateurs opérationnels. Les données doivent être disponibles pour les équipes analytiques tout en respectant des contraintes de sécurité, de traçabilité et de gouvernance. Cette exigence justifie l'intérêt d'une plateforme de données structurée, automatisée et observable.

1.3 Cadre du projet

Le projet consiste à concevoir et déployer une **Cloud Data Platform** sur AWS EKS, adaptée aux besoins d'un environnement financier manipulant des données opérationnelles, analytiques et décisionnelles. La plateforme vise à couvrir le cycle de vie complet des données : ingestion depuis les systèmes sources, stockage dans un Lakehouse, transformation en couches analytiques, orchestration des traitements, exposition SQL/BI et supervision.

L'organisation métier du projet, illustrée dans la figure 1.7, met en relation les besoins fonctionnels, les équipes utilisatrices et les briques techniques chargées de produire des données fiables. Elle sert de point d'appui pour comprendre comment la plateforme transforme des sources opérationnelles en indicateurs exploitables par les métiers.

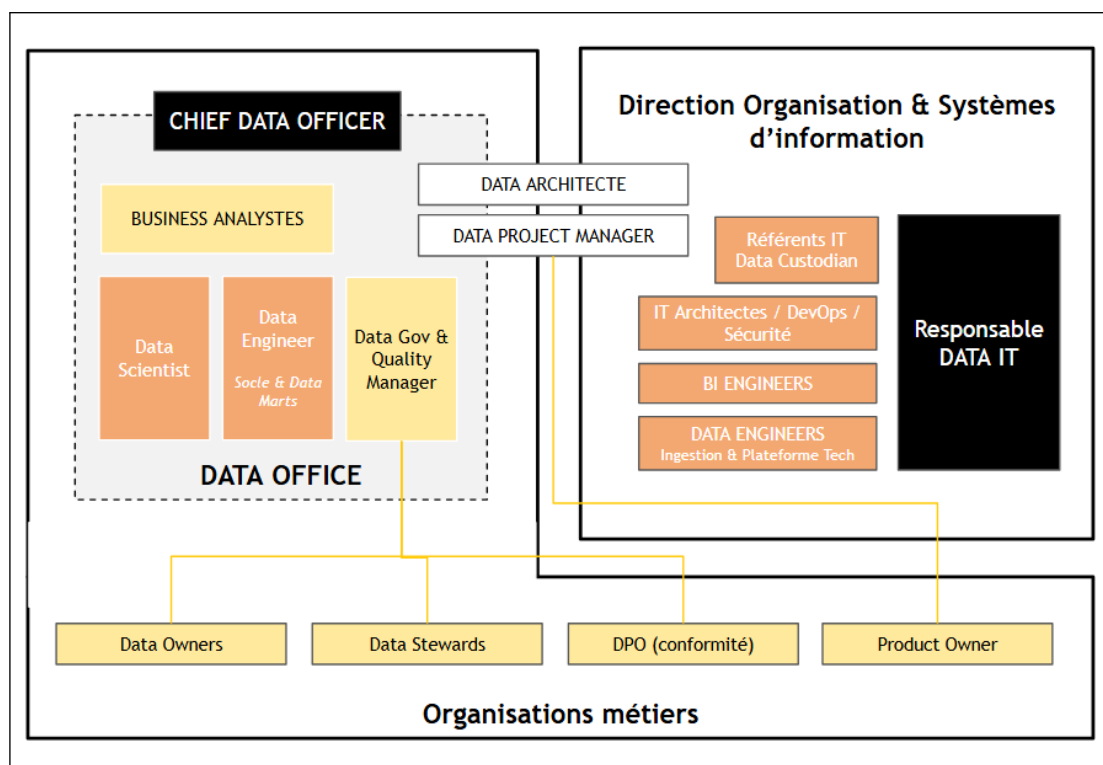


FIGURE 1.7 – Organisation métier autour de la plateforme data

Le cadrage fonctionnel et technique retient une approche progressive : bâtir d’abord un socle cloud sécurisé et reproductible, puis y intégrer les services nécessaires au pipeline data, à la gouvernance, à l’observabilité et à la consommation des données par les équipes BI. Les livrables principaux regroupent ainsi les éléments nécessaires à la mise en place, à l’exploitation et à la démonstration de la plateforme :

- les modules Terraform de la landing zone AWS ;
- les manifests Kubernetes et Helm [23] des différentes phases ;
- les scripts de déploiement, destruction, préparation des secrets et construction d’image ;
- les DAGs Airflow et jobs Spark/dbt ;
- la documentation d’exploitation, de traçabilité et de maintenance.

1.4 Problématique

Dans un contexte financier, les données proviennent de systèmes hétérogènes : bases opérationnelles, applications métiers, fichiers bureautiques et flux applicatifs. Cette diversité complique leur centralisation, leur historisation et leur exploitation analytique. Les équipes métier ont besoin de données fiables, sécurisées et disponibles pour le pilotage, tandis que les équipes techniques doivent garantir la traçabilité des traitements, la reproductibilité des déploiements et la supervision de bout en bout.

La problématique du projet peut donc être formulée ainsi :

Comment concevoir une plateforme data cloud, sécurisée, observable et extensible, capable d’ingérer des sources hétérogènes, de structurer les données dans un Lakehouse et de les exposer aux usages BI et analytiques, tout en assurant la traçabilité, la gouvernance et la maîtrise opérationnelle des traitements ?

Cette problématique se décline en plusieurs axes complémentaires, représentés dans la figure 1.8.

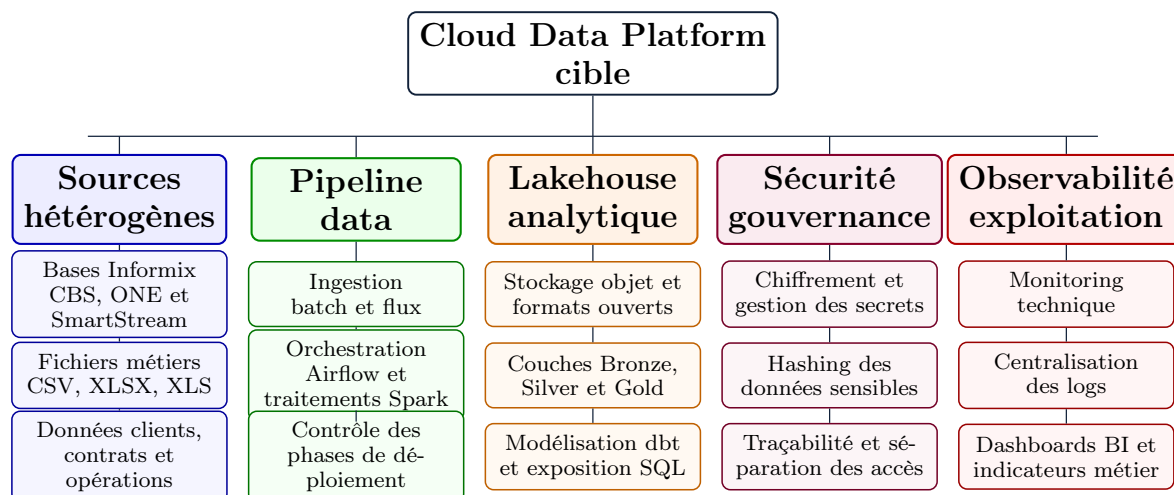


FIGURE 1.8 – Taxonomie de la problématique data du projet

1.5 Méthodologie de conduite

La démarche adoptée suit une approche itérative inspirée de Scrum. Le projet a été découpé en phases techniques cohérentes, chacune produisant un incrément exploitable : socle cloud, stockage, ingestion, compute, serving, exposition, observabilité et automatisation.

La figure 1.9 rappelle le cycle itératif retenu, tandis que le tableau 1.4 associe chaque sprint à ses livrables principaux.

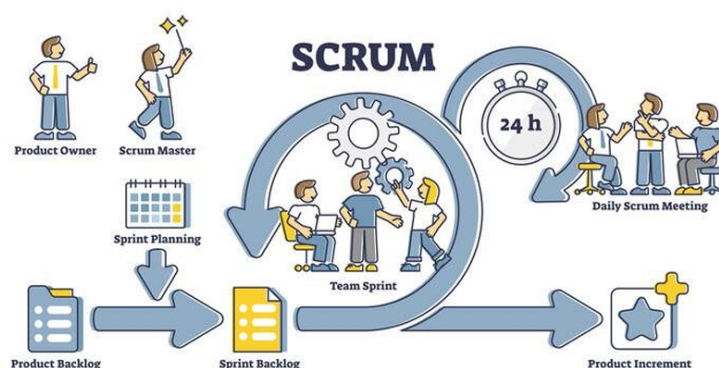


FIGURE 1.9 – Démarche agile appliquée au projet

TABLE 1.4 – Planification synthétique du projet

Sprint	Phase	Livrables principaux
Sprint 1	Socle cloud	VPC, EKS, KMS, IAM, backend R2, CI Terraform, namespaces Kubernetes.
Sprint 2	Stockage	MinIO, CloudNativePG [24], Hive Metastore, secrets, stockage gp3.
Sprint 3	Ingestion	Kafka Strimzi, topics, KafkaConnect, Debezium, NiFi.
Sprint 4	Compute	Image Spark Iceberg, SparkApplication, dbt, Airflow et DAGs.
Sprint 5	Serving / sécurité	Dremio, source Hive, Cloudflare Tunnel [25] et exposition des interfaces.
Sprint 6	Observabilité	Prometheus, Grafana, Fluent Bit, OpenObserve, ServiceMonitors et documentation finale.

1.6 Planification du projet

La figure 1.10 détaille la planification temporelle du projet. Elle montre l’enchaînement des travaux de cadrage, de conception, d’implémentation et de validation jusqu’à la soutenance du 26 juin 2026.

Diagramme de Gantt du projet

Planification de la Cloud Data Platform – mars à juin 2026



FIGURE 1.10 – Diagramme de Gantt du projet

1.7 Apports attendus

Le projet apporte une base technique réutilisable pour construire une plateforme data moderne :

- une infrastructure cloud décrite par code et reconstruisible ;
- une architecture Lakehouse compatible avec les standards open source ;
- une organisation par phases qui facilite le déploiement progressif ;
- une documentation permettant d’expliquer les choix techniques et les limites du MVP ;
- une trajectoire claire vers l’industrialisation : GitOps, monitoring avancé, data quality et BI métier.

Conclusion du chapitre

Ce chapitre a permis de situer le projet dans son environnement professionnel. SEVENAPP intervient comme organisme d’accueil et comme cadre d’accompagnement technique, tandis qu’EQDOM constitue le contexte métier de la mission. La présentation du client a mis en évidence les contraintes propres au secteur financier : fiabilité des traitements, sécurité des accès, traçabilité des données et disponibilité d’indicateurs exploitables pour le pilotage.

Le cadrage a également précisé la problématique centrale du projet : construire une plateforme capable de centraliser des sources hétérogènes, de structurer les données selon une logique Lakehouse et de les exposer progressivement aux usages analytiques. La taxonomie proposée montre que cette problématique ne se limite pas au stockage. Elle englobe l’ingestion, la transformation, la gouvernance, l’observabilité et l’exploitation opérationnelle.

Enfin, l’organisation itérative et le diagramme de Gantt ont permis de relier les objectifs aux différentes étapes de réalisation, depuis le cadrage jusqu’à la validation finale prévue pour la soutenance du 26 juin 2026. Le projet dispose ainsi d’un périmètre cohérent, d’une démarche de conduite explicite et d’une trajectoire d’industrialisation identifiable.

Le chapitre suivant présente les concepts techniques nécessaires pour comprendre les choix retenus : architecture Lakehouse, modèle médaillon, Kubernetes, operators, ingestion batch et streaming, transformation distribuée, Infrastructure as Code et observabilité.

Chapitre 2 : ÉTAT DE L'ART

Ce chapitre présente les fondements techniques sur lesquels repose la solution. Il ne s'agit pas d'une simple liste d'outils, mais d'une analyse des concepts qui structurent les plateformes de données modernes : Lakehouse, architecture médaillon, Kubernetes, operators, ingestion temps réel, transformation distribuée, virtualisation SQL et observabilité. L'objectif est de relier chaque concept à son rôle concret dans la plateforme cible.

La figure 2.1 introduit la vue d'ensemble utilisée comme fil conducteur dans ce chapitre.

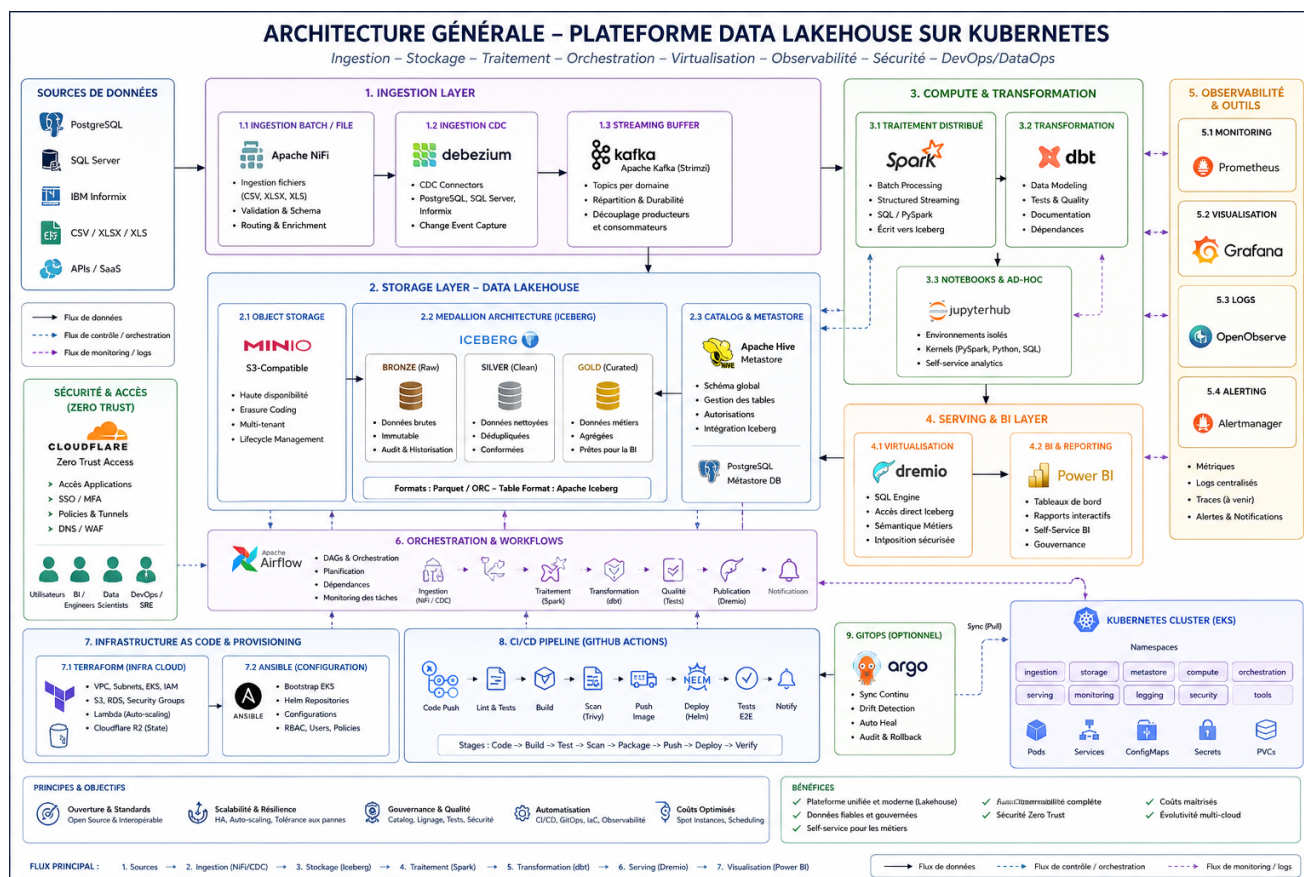


FIGURE 2.1 – Architecture générale de la plateforme Data Lakehouse sur Kubernetes

La figure 2.1 synthétise l'architecture cible du projet. Les sources de données alimentent une couche d'ingestion composée de NiFi [26], Debezium et Kafka. Les données sont ensuite stockées dans un Lakehouse fondé sur MinIO, Iceberg et Hive Metastore. Les traitements Spark et dbt produisent les couches analytiques, Airflow orchestre les workflows, Dremio expose les données en SQL et Power BI [27] les consomme côté métier. Les briques d'observabilité, de sécurité, d'Infrastructure as Code et de CI/CD permettent de rendre l'ensemble exploitable, reproductible et gouverné.

2.1 Évolution des plateformes de données

Les premières architectures décisionnelles reposaient principalement sur des entrepôts de données centralisés. Ces solutions offraient de bonnes performances analytiques, mais elles étaient coûteuses, rigides face aux données semi-structurées et difficiles à faire évoluer rapidement. Les data lakes ont ensuite permis de stocker de grands volumes de données brutes à moindre coût, généralement sur stockage objet. En contrepartie, ils ont introduit de nouveaux défis : gouvernance insuffisante, qualité variable, absence de transactions fiables et difficulté à exposer les données aux utilisateurs métiers.

Les architectures **Lakehouse** répondent à cette tension en combinant la flexibilité du data lake avec certaines garanties du data warehouse. Elles s'appuient sur des formats de tables ouverts, comme Apache Iceberg, qui apportent un catalogue, une évolution de schéma, des snapshots, une meilleure isolation des écritures et une exploitation efficace par plusieurs moteurs de calcul. Dans le cadre du projet, ce modèle est pertinent car il permet d'unifier stockage brut, transformation analytique et consommation BI sans multiplier les silos.

La figure 2.2 résume cette évolution, puis le tableau 2.1 compare les propriétés des trois modèles.

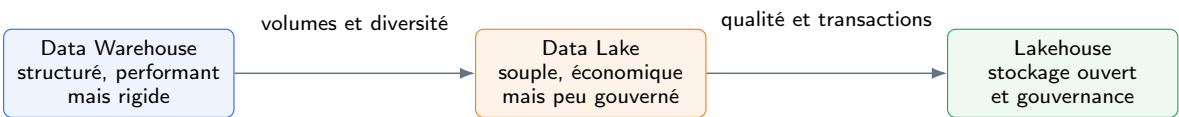


FIGURE 2.2 – Évolution des architectures de données vers le Lakehouse

TABLE 2.1 – Comparaison synthétique des architectures data

Critère	Data Warehouse	Data Lake	Lakehouse
Type de données	Structurées	Brutes, variées	Structurées et semi-structurées
Coût de stockage	Élevé	Faible	Faible à modéré
Gouvernance	Forte	Variable	Renforcée par catalogue et formats ouverts
Évolution de schéma	Contrôlée mais rigide	Souple mais risquée	Souple avec métadonnées transactionnelles
Cas d'usage	BI classique	Archivage, exploration	BI, ML, streaming, batch

2.2 Architecture Lakehouse et modèle médaille

Le modèle **médaille** organise les données selon leur niveau de qualité et de préparation. La couche Bronze conserve les données brutes et historisées, ce qui permet de rejouer les traitements en cas d'erreur. La couche Silver applique les règles de nettoyage, de typage, de déduplication et de standardisation. La couche Gold produit des tables orientées usages métiers : indicateurs, agrégats, référentiels analytiques et jeux de données prêts pour la BI.

Dans ce projet, Apache Iceberg fournit la structure transactionnelle du Lakehouse, tandis que MinIO apporte le stockage objet compatible S3. Hive Metastore joue le rôle de catalogue central afin que Spark, dbt et Dremio partagent la même représentation des tables. Cette séparation entre stockage, catalogue et moteurs de calcul évite l'enfermement dans un entrepôt propriétaire et facilite l'évolution de la plateforme.

La figure 2.3 illustre la progression de la donnée entre les couches Bronze, Silver et Gold ainsi que le rôle transverse du catalogue.

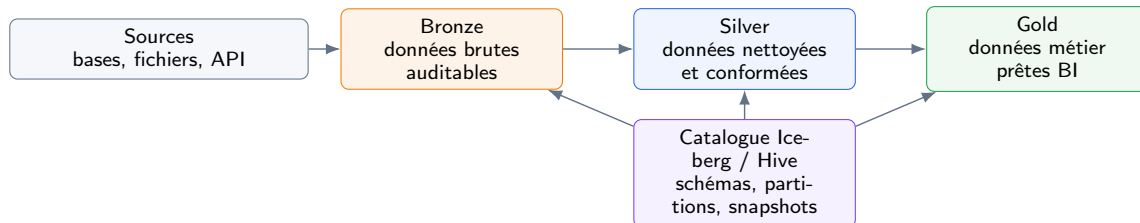


FIGURE 2.3 – Organisation du Lakehouse selon le modèle médaillon

2.3 Kubernetes et approche cloud-native

Kubernetes s'est imposé comme standard d'orchestration des applications conteneurisées. Dans une plateforme data, il permet de déployer de manière homogène des composants hétérogènes : bases de données, brokers Kafka, jobs Spark, orchestrateurs, services SQL et outils d'observabilité. Chaque composant est empaqueté sous forme de conteneur, puis exécuté dans un cluster qui gère le placement, le redémarrage, l'exposition réseau et la montée en charge.

L'intérêt de Kubernetes repose sur quatre axes. La **portabilité** permet d'adapter la plateforme à AWS, à un autre cloud ou à un environnement on-premise. L'**isolation** structure les responsabilités grâce aux namespaces, RBAC, quotas et NetworkPolicies. L'**élasticité** permet de dimensionner les traitements selon la charge. L'**automatisation** facilite l'exploitation au moyen d'operators et de manifests déclaratifs.

La figure 2.4 présente la vue infrastructure complète ; la figure 2.5 en extrait ensuite le rôle d'isolation et d'orchestration joué par Kubernetes.

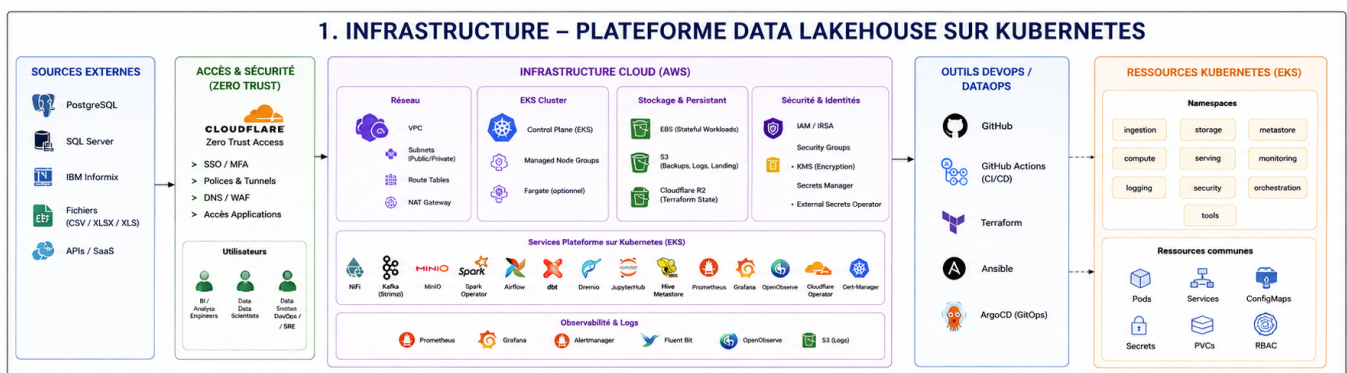


FIGURE 2.4 – Architecture infrastructure de la plateforme Data Lakehouse

La figure 2.4 détaille la vue infrastructure de la plateforme. Les sources externes et les utilisateurs accèdent à l'environnement au travers d'une couche de sécurité Zero Trust portée

par Cloudflare. Le socle cloud AWS regroupe le réseau, le cluster EKS, le stockage persistant, les identités IAM/IRSA, le chiffrement KMS et la gestion des secrets. Les services data sont ensuite déployés dans Kubernetes : ingestion, Kafka, MinIO, Spark, Airflow, dbt, Dremio, Hive Metastore et observabilité. La partie DevOps/DataOps montre que GitHub Actions, Terraform, Ansible et GitOps servent à automatiser le provisionnement, les déploiements et la maintenance. Enfin, la zone EKS met en évidence les namespaces et ressources communes, ce qui illustre la séparation des responsabilités techniques dans le cluster.

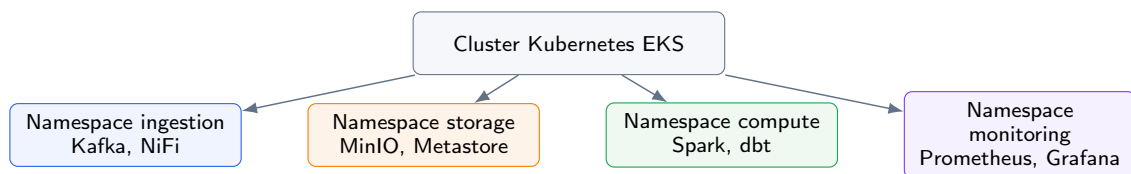


FIGURE 2.5 – Rôle de Kubernetes dans l'isolation et l'orchestration des composants

2.4 Pattern Operator

Un **operator** Kubernetes [28] étend l'API du cluster au moyen de Custom Resource Definitions (CRD). Au lieu de déployer manuellement chaque objet technique, l'équipe décrit l'état souhaité dans une ressource déclarative. Le contrôleur de l'operator observe ensuite l'état réel, détecte les écarts et applique les actions nécessaires : création de pods, configuration, sauvegardes, montée de version ou réparation.

Cette logique est particulièrement utile pour les composants stateful et complexes. Déployer Kafka, PostgreSQL, MinIO ou Spark à la main demande de maîtriser de nombreuses règles d'exploitation. Les operators encapsulent cette expertise et rendent les déploiements plus reproductibles. Ils ne remplacent pas la compréhension technique, mais réduisent fortement les opérations répétitives et les risques d'erreur.

La figure 2.6 décrit cette boucle de réconciliation. Le tableau 2.2 précise ensuite les operators mobilisés et la couche prise en charge par chacun.

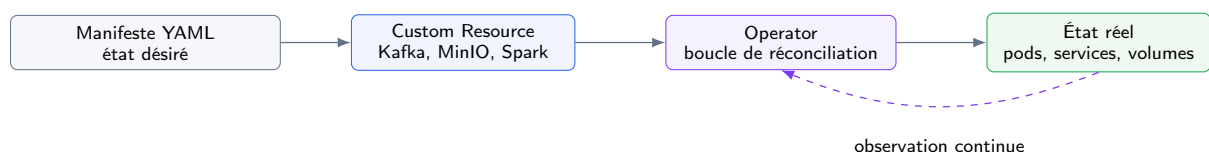


FIGURE 2.6 – Principe de fonctionnement du pattern Operator

TABLE 2.2 – Operators mobilisés dans le projet

Operator	Rôle	Couche
CloudNativePG	Déploiement et gestion de clusters PostgreSQL	Source / Metastore
MinIO Operator	Gestion d'un tenant de stockage objet compatible S3	Stockage
Strimzi	Déploiement de Kafka, topics et KafkaConnect	Ingestion
Spark Operator	Soumission de traitements Spark comme ressources Kubernetes	Calcul
Cloudflare Operator	Publication contrôlée des services via tunnels Zero Trust	Sécurité / Exposition

2.5 Ingestion batch et streaming

Une plateforme de données doit couvrir plusieurs modes d'ingestion. Les fichiers CSV/XLSX sont fréquents dans les processus opérationnels et nécessitent une ingestion batch robuste : contrôle du format, validation, routage et enrichissement. Les bases transactionnelles, quant à elles, doivent pouvoir alimenter la plateforme sans exports complets répétés. La **Change Data Capture** répond à ce besoin en capturant les insertions, mises à jour et suppressions directement depuis le journal de transaction.

Le couple **Kafka** + **Debezium** constitue une solution standard pour ce cas d'usage. Kafka sert de buffer durable et découple la source des consommateurs. Debezium transforme les changements de PostgreSQL ou Informix en événements exploitables. Apache NiFi complète l'ensemble en offrant une interface orientée flux pour les scénarios fichier ou semi-structurés. Les données convergent ensuite vers la couche Bronze, où elles restent rejouables et traçables.

La figure 2.7 met en parallèle les chemins batch et streaming avant leur convergence vers Bronze.

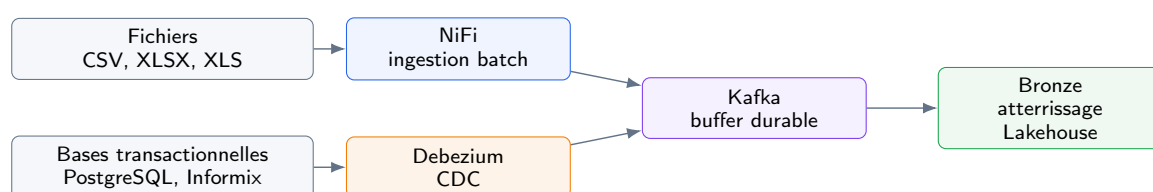


FIGURE 2.7 – Modes d'ingestion batch et streaming vers la couche Bronze

2.6 Transformation distribuée et orchestration

Apache Spark est retenu pour les traitements distribués, notamment l'écriture de tables Iceberg et la consommation de flux Kafka. Sa capacité à paralléliser les traitements permet d'absorber des volumes importants tout en conservant une logique de transformation programmable en SQL ou PySpark. dbt complète Spark en introduisant une approche déclarative des transformations SQL, avec un modèle de projet versionné, testable et compréhensible par les profils data.

Airflow joue le rôle d'orchestrateur. Il permet de planifier les traitements, de suivre leur exécution et de structurer les dépendances entre ingestion Bronze, transformation Silver et production Gold. Dans la solution, les DAGs Airflow soumettent des ressources SparkApplication, ce qui conserve une séparation claire entre orchestration et calcul : Airflow décide quand exécuter, Spark exécute le traitement, dbt formalise une partie des transformations analytiques.

La figure 2.8 présente le pipeline complet, tandis que la figure 2.9 isole la chaîne de transformation et les responsabilités de chaque outil.

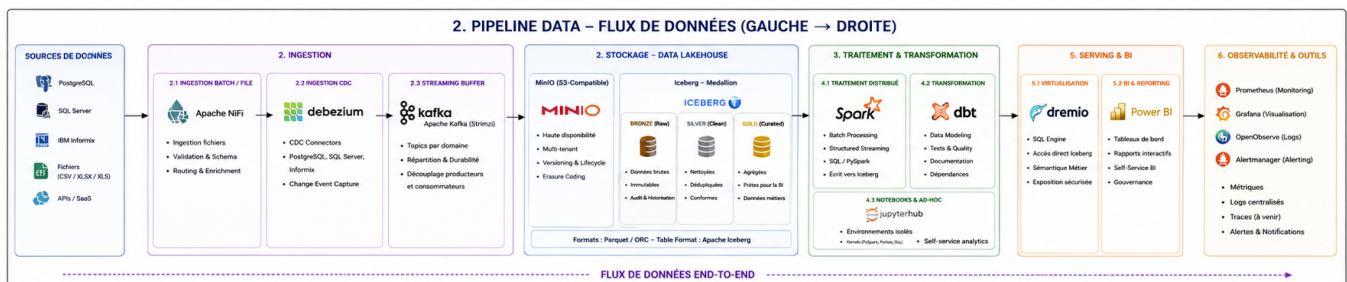


FIGURE 2.8 – Pipeline data end-to-end de la plateforme

La figure 2.8 présente le flux fonctionnel de bout en bout. Les sources de données, qu'elles soient relationnelles, fichiers ou API, alimentent d'abord la couche d'ingestion. NiFi couvre les flux batch et fichiers, Debezium capture les changements des bases via CDC, puis Kafka sert de tampon durable et découple les producteurs des consommateurs. Les données arrivent ensuite dans le Lakehouse, où MinIO fournit le stockage objet et Iceberg structure les couches Bronze, Silver et Gold. La transformation s'effectue avec Spark pour les traitements distribués et dbt pour la modélisation SQL, les tests et la documentation. Enfin, Dremio expose les tables au travers d'une couche SQL et Power BI les consomme pour les tableaux de bord. Les outils d'observabilité suivent l'ensemble du parcours afin de surveiller les métriques, les logs, les alertes et la traçabilité.



FIGURE 2.9 – Chaîne de transformation et d'orchestration des traitements

2.7 Virtualisation SQL et Business Intelligence

Dremio est utilisé comme couche de virtualisation SQL. Son rôle est de permettre l'interrogation des tables Iceberg sans déplacer les données vers un entrepôt propriétaire. Les analystes peuvent ainsi accéder aux données au moyen d'un moteur SQL, tandis que le stockage principal reste dans le Lakehouse. Cette approche réduit les duplications, simplifie la gouvernance et facilite l'exposition progressive des jeux de données Gold.

La couche BI, représentée par Power BI dans l'architecture cible, consomme les datasets publiés par Dremio. Les rapports peuvent être construits à partir de vues sémantiques, d'agrégats Gold ou de requêtes contrôlées. Le point essentiel est la séparation des responsabilités : le Lakehouse conserve les données, Dremio sert de couche d'accès SQL et Power BI porte la visualisation métier.

La figure 2.10 synthétise cette séparation entre stockage Gold, accès SQL, vues sémantiques et restitution BI.



FIGURE 2.10 – Exposition SQL et consommation BI des données Gold

2.8 Infrastructure as Code et CI/CD

L'Infrastructure as Code constitue un pilier du projet. Terraform décrit les ressources cloud : réseau, cluster, IAM, clés KMS, ECR et scaling. Le choix d'un backend Cloudflare R2 pour l'état Terraform permet de séparer la gestion de l'état du compte AWS déployé, ce qui réduit le couplage opérationnel. Les manifests Kubernetes, charts Helm [23] et configurations applicatives complètent cette logique déclarative côté cluster.

La CI/CD avec GitHub Actions [29] permet d'exécuter les contrôles, les plans Terraform et les déploiements. L'authentification OIDC évite l'usage de clés AWS statiques longues durées, ce qui constitue une bonne pratique de sécurité. L'ensemble forme une chaîne reproductible : le code décrit l'infrastructure, la pipeline vérifie les changements, puis le déploiement applique les versions validées.

La figure 2.11 illustre ce chemin déclaratif depuis le dépôt Git jusqu'à la plateforme EKS.

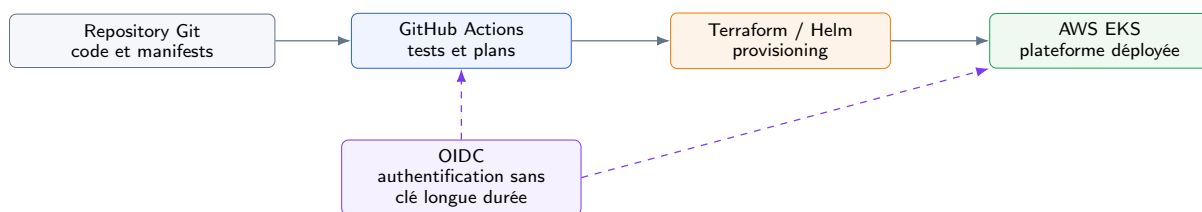


FIGURE 2.11 – Chaîne Infrastructure as Code et CI/CD

2.9 Observabilité

Une plateforme de données n'est exploitable que si elle est observable. Les métriques indiquent l'état technique des composants : CPU, mémoire, disponibilité des pods, performances Kafka, état PostgreSQL ou exécution des jobs Spark. Les logs donnent le détail des événements applicatifs et facilitent l'analyse des incidents. Les dashboards et alertes permettent enfin d'identifier rapidement les défaillances et d'enclencher les actions correctives.

Le projet s'appuie sur Prometheus, Grafana, Fluent Bit et OpenObserve. Prometheus collecte les métriques, Grafana les visualise, Fluent Bit transporte les logs et OpenObserve centralise leur exploration. Cette combinaison couvre à la fois le monitoring technique, la traçabilité des traitements et le suivi opérationnel des services exposés.

La figure 2.12 représente les deux flux complémentaires, métriques et logs, qui alimentent les outils de diagnostic et les alertes.

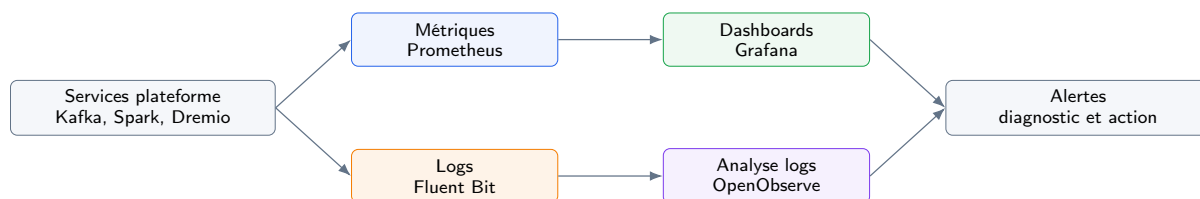


FIGURE 2.12 – Piliers d’observabilité de la plateforme

Conclusion du chapitre

Ce chapitre a présenté les fondements techniques nécessaires à la conception d’une plateforme de données moderne. L’évolution du Data Warehouse vers le Data Lake, puis vers le Lakehouse, répond à un besoin de compromis entre flexibilité, coût de stockage et gouvernance. Dans ce projet, le choix du Lakehouse permet de conserver des données brutes sur stockage objet tout en introduisant des garanties utiles pour les traitements analytiques : catalogue partagé, évolution de schéma, historisation et formats de tables ouverts avec Apache Iceberg.

Le modèle médaillon complète cette architecture en organisant la progression de la qualité des données. La couche Bronze conserve une trace proche de la source, la couche Silver applique les transformations de normalisation et la couche Gold produit des tables adaptées aux usages métier. Cette séparation facilite le rejeu des traitements, le contrôle de la qualité et l’exposition de données stabilisées aux outils de Business Intelligence.

L’étude de Kubernetes et du pattern Operator a ensuite montré comment rendre cette plateforme déployable et exploitable. Kubernetes fournit un socle homogène pour isoler les composants, piloter les ressources et automatiser les redémarrages. Les operators ajoutent une logique déclarative adaptée aux services complexes comme PostgreSQL, Kafka, MinIO ou Spark. La plateforme peut ainsi être décrite par code et reconstruite de manière reproductible.

La chaîne data repose également sur la complémentarité des outils. NiFi couvre les flux batch et semi-structurés, Debezium capture les changements des bases transactionnelles, Kafka découple les producteurs et les consommateurs, Spark exécute les traitements distribués et dbt structure les transformations SQL. Airflow orchestre ces étapes, tandis que Dremio expose les tables Gold à la BI sans déplacer les données hors du Lakehouse.

Enfin, l’Infrastructure as Code, la CI/CD et l’observabilité ne sont pas des éléments secondaires. Terraform, Helm et GitHub Actions rendent les déploiements contrôlables ; Prometheus, Grafana, Fluent Bit et OpenObserve permettent de surveiller les métriques, les logs et les incidents. L’état de l’art aboutit donc à une architecture cohérente, fondée sur des standards cloud-native et sur une séparation claire des responsabilités.

Ces principes constituent le cadre de référence du projet. Le chapitre suivant les traduit en besoins fonctionnels, exigences non fonctionnelles, acteurs, scénarios de traitement et critères d’acceptation.

Chapitre 3 : ANALYSE ET SPÉCIFICATION DES BESOINS

Ce chapitre formalise la problématique, les objectifs et les exigences de la Cloud Data Platform. L’analyse se base sur le cahier des charges du projet, les guides techniques fournis et l’état réel du dépôt d’implémentation. Elle vise à établir un lien clair entre les besoins exprimés, les choix de conception et les artefacts livrés.

3.1 Problématique

Les systèmes d’information financiers produisent des données à forte valeur, mais souvent réparties dans plusieurs environnements : bases opérationnelles, exports bureautiques et fichiers métier. Dans le périmètre du projet, les sources à prendre en charge sont clairement identifiées : **PostgreSQL pour la base ONE, Informix CBS, SmartStream**, ainsi que des **fichiers métiers CSV, XLS et XLSX**. Cette dispersion entraîne plusieurs difficultés :

- la consolidation des données nécessite des traitements manuels ou semi-automatisés ;
- les transformations ne sont pas toujours versionnées, testées ou rejouables ;
- les données brutes et les données préparées ne sont pas clairement séparées ;
- les incidents de pipeline sont difficiles à diagnostiquer sans centralisation des logs et métriques ;
- le passage d’un prototype à un socle industrialisé demande une infrastructure reproductible.

La problématique peut donc être formulée ainsi :

Comment concevoir et mettre en œuvre une plateforme de données cloud-native, reproductible et sécurisée, capable d’ingérer des données batch et streaming, de les structurer selon une architecture Lakehouse médaillon, puis de les exposer à des usages analytiques tout en assurant supervision, traçabilité et maîtrise des coûts ?

3.2 Objectifs du projet

Le projet poursuit quatre objectifs complémentaires :

1. **Objectif infrastructure** : créer une landing zone AWS/EKS modulaire et automatisée par Terraform.
2. **Objectif data engineering** : mettre en place un pipeline Bronze–Silver–Gold fondé sur MinIO, Iceberg, Spark et dbt.

3. **Objectif opérationnel** : fournir des scripts de déploiement, de destruction, de bootstrap et de validation par phase.
4. **Objectif gouvernance** : documenter les choix d'architecture, les exigences, la sécurité, les limites et les perspectives d'industrialisation.

3.3 Périmètre fonctionnel

Le périmètre fonctionnel de la plateforme couvre deux familles de sources. La première correspond aux bases de données opérationnelles, dont les changements doivent pouvoir être capturés ou ingérés sans imposer d'exports manuels répétés. La seconde correspond aux fichiers métiers, souvent utilisés dans les processus de reporting ou de consolidation ponctuelle.

Le tableau 3.1 recense les sources retenues et précise leur rôle. Le tableau 3.2 traduit ensuite ce périmètre en exigences fonctionnelles vérifiables.

TABLE 3.1 – Sources de données retenues dans le périmètre

Famille	Source	Rôle dans la plateforme
Base opérationnelle	PostgreSQL – base ONE	Source applicative ingérée vers la couche Bronze, avec possibilité de capture des changements via CDC.
Base opérationnelle	Informix – base Informix CBS	Source applicative à intégrer au Lakehouse pour centraliser les données opérationnelles et analytiques.
Base opérationnelle	SmartStream	Source historique ou métier à connecter à la plateforme selon les besoins d'ingestion.
Fichiers métiers	CSV, XLS, XLSX	Exports métier ou fichiers de consolidation intégrés en batch vers la couche Bronze.

TABLE 3.2: Exigences fonctionnelles principales

ID	Exigence	Priorité
EF-01	Provisionner une infrastructure AWS comprenant VPC, EKS, IAM, KMS, ECR et backend Terraform distant.	Essentielle
EF-02	Créer les namespaces Kubernetes de la plateforme avec isolation réseau, quotas et stockage chiffré.	Essentielle
EF-03	Déploier une couche de stockage Lakehouse basée sur MinIO, Iceberg et Hive Metastore.	Essentielle
EF-04	Prendre en charge les sources bases de données du périmètre : PostgreSQL pour la base ONE, Informix CBS et SmartStream.	Essentielle

ID	Exigence	Priorité
EF-05	Capturer ou ingérer les changements des bases sources, notamment PostgreSQL/ONE via Debezium, et publier les événements dans Kafka lorsque le mode CDC est retenu.	Essentielle
EF-06	Ingérer des fichiers métiers CSV, XLS et XLSX vers la couche Bronze à l'aide de traitements batch.	Importante
EF-07	Transformer les données Bronze en Silver puis Gold avec dbt-spark.	Essentielle
EF-08	Orchestrer les traitements au moyen de DAGs Airflow versionnés.	Essentielle
EF-09	Exposer les tables analytiques via Dremio pour les requêtes SQL et la BI.	Importante
EF-10	Publier les interfaces web par Cloudflare Zero Trust.	Souhaitable
EF-11	Collecter les métriques, logs et dashboards nécessaires à l'exploitation.	Essentielle
EF-12	Préparer une trajectoire GitOps avec ArgoCD [30].	Optionnelle

3.4 Exigences non fonctionnelles

Le tableau 3.3 complète les fonctionnalités attendues par les contraintes de qualité qui guident la conception : sécurité, reproductibilité, maintenabilité, scalabilité, coût, observabilité et traçabilité.

TABLE 3.3 – Exigences non fonctionnelles

Axe	Exigence
Sécurité	Principe du moindre privilège, IRSA, KMS, secrets hors Git, accès EKS restreint, exposition par tunnel sécurisé.
Reproductibilité	Infrastructure et composants décrits par code : Terraform, Helm, manifests Kubernetes et scripts Bash.
Maintenabilité	Organisation par phases, documentation d'exploitation, séparation entre modules Terraform et manifests applicatifs.
Scalabilité	Node groups spécialisés, capacité de montée en charge du compute et architecture extensible à plusieurs sources.
Coût	Dimensionnement dev , destruction automatisée, NAT unique en MVP et scaling planifié du node group compute.
Observabilité	Supervision des ressources, logs centralisés, dashboards et alertes sur les services critiques.
Traçabilité	Correspondance entre cahier des charges, exigences, décisions d'architecture et fichiers du dépôt.

3.5 Acteurs et responsabilités

Le tableau 3.4 identifie les principaux acteurs du système et clarifie la répartition des responsabilités entre construction, exploitation, validation et consommation des données.

TABLE 3.4 – Acteurs du système

Acteur	Responsabilités
Data Engineer	Déploiement de la plateforme, ingestion des sources, développement des jobs Spark/dbt, maintenance des DAGs.
Analyste BI	Exploration des tables Gold via Dremio et création de tableaux de bord métier.
Administrateur Cloud/SRE	Supervision du cluster, gestion des incidents, coûts, accès, logs et métriques.
Encadrant technique	Validation de l'architecture, revue des choix, contrôle de l'alignement avec le cahier des charges.

3.5.1 Diagrammes de compréhension fonctionnelle et technique

Les figures 3.1 et 3.2 complètent la lecture des responsabilités en montrant comment les acteurs, les sources et les composants techniques interagissent dans la plateforme. Elles servent de transition entre l'analyse des besoins et le scénario de référence du pipeline data.

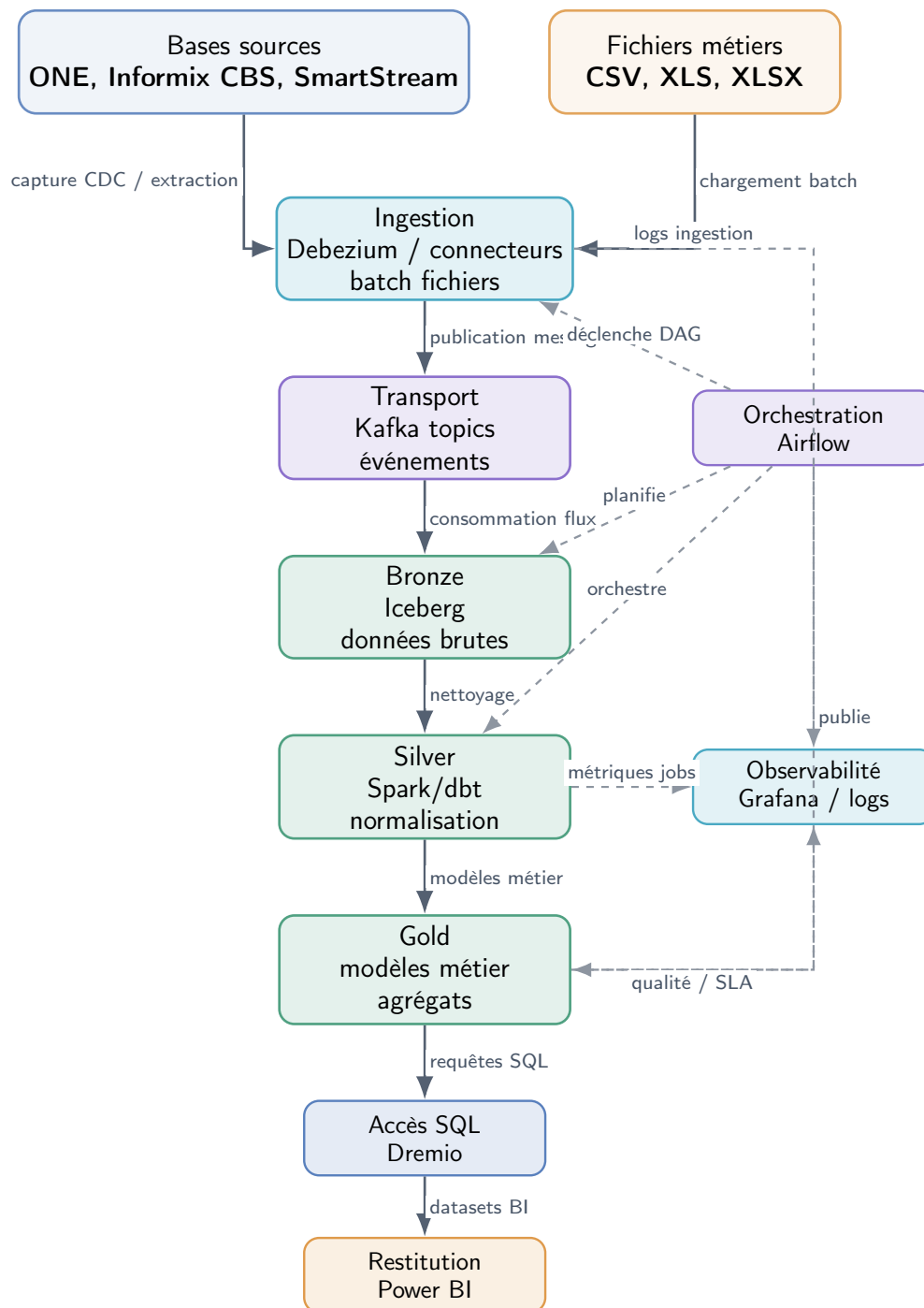


FIGURE 3.1 – Diagramme de flux de données de la plateforme

Ce flux met en évidence deux chemins d'entrée : les bases opérationnelles et les fichiers métiers. Les flèches pleines représentent le chemin principal de la donnée : capture ou extraction, publication dans Kafka, écriture dans Bronze, transformation vers Silver, publication Gold, puis exposition SQL dans Dremio et Power BI. Les flèches en pointillés représentent les relations de pilotage et de supervision : Airflow orchestre les traitements, tandis que Grafana et les logs suivent l'état des jobs, la qualité et les SLA.

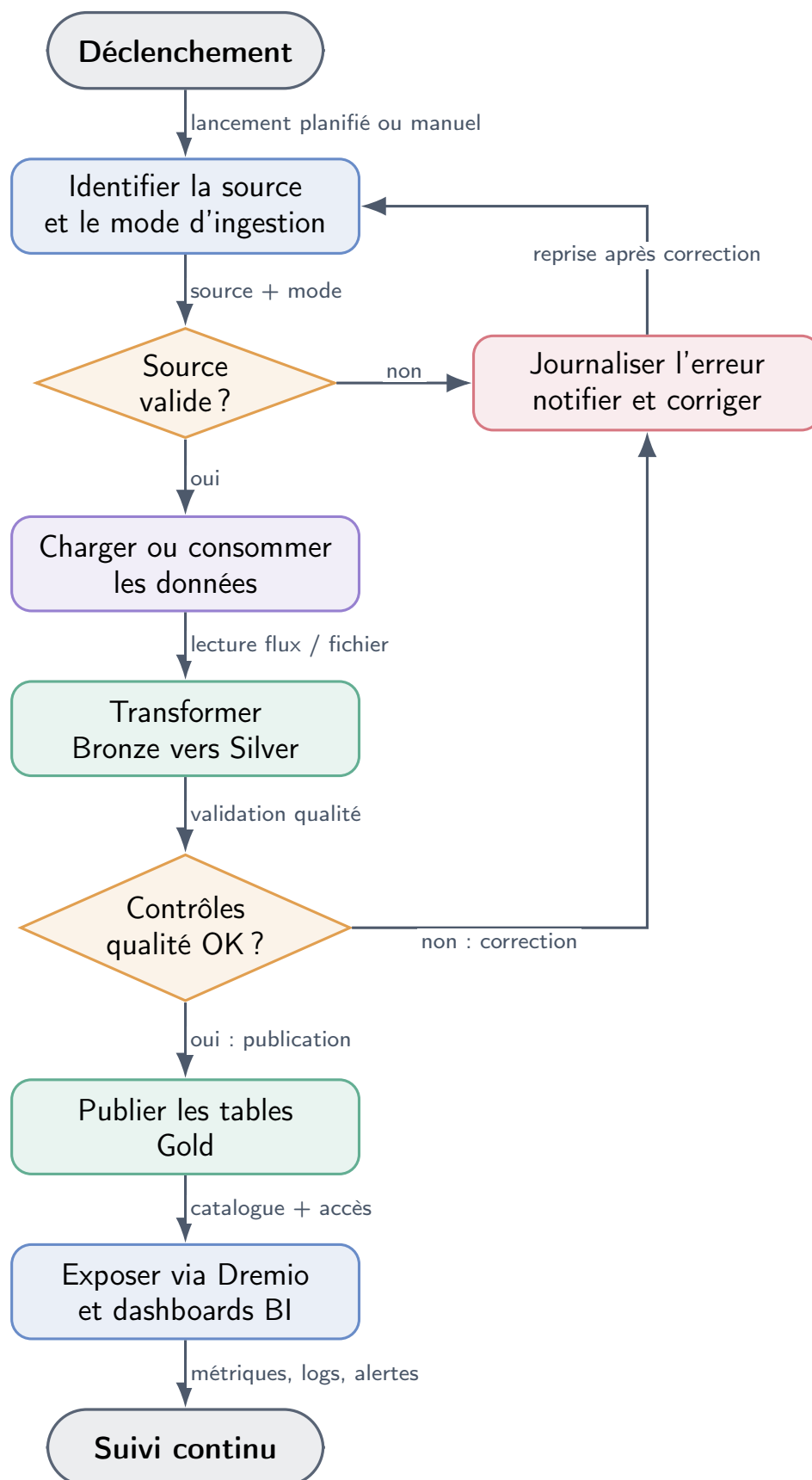


FIGURE 3.2 – Diagramme d'activités du pipeline data

Le diagramme d'activités décrit la logique de traitement attendue. Les relations indiquées sur les flèches précisent le rôle de chaque transition : lancement planifié ou manuel, qualification de la source, lecture du flux ou du fichier, contrôle qualité, publication dans le catalogue et supervision continue. Les retours vers la correction représentent les reprises nécessaires lorsqu'une source ou une transformation n'est pas conforme.

La figure 3.3 complète les deux diagrammes précédents par une vue technique de l'infrastructure cible et de ses dépendances.

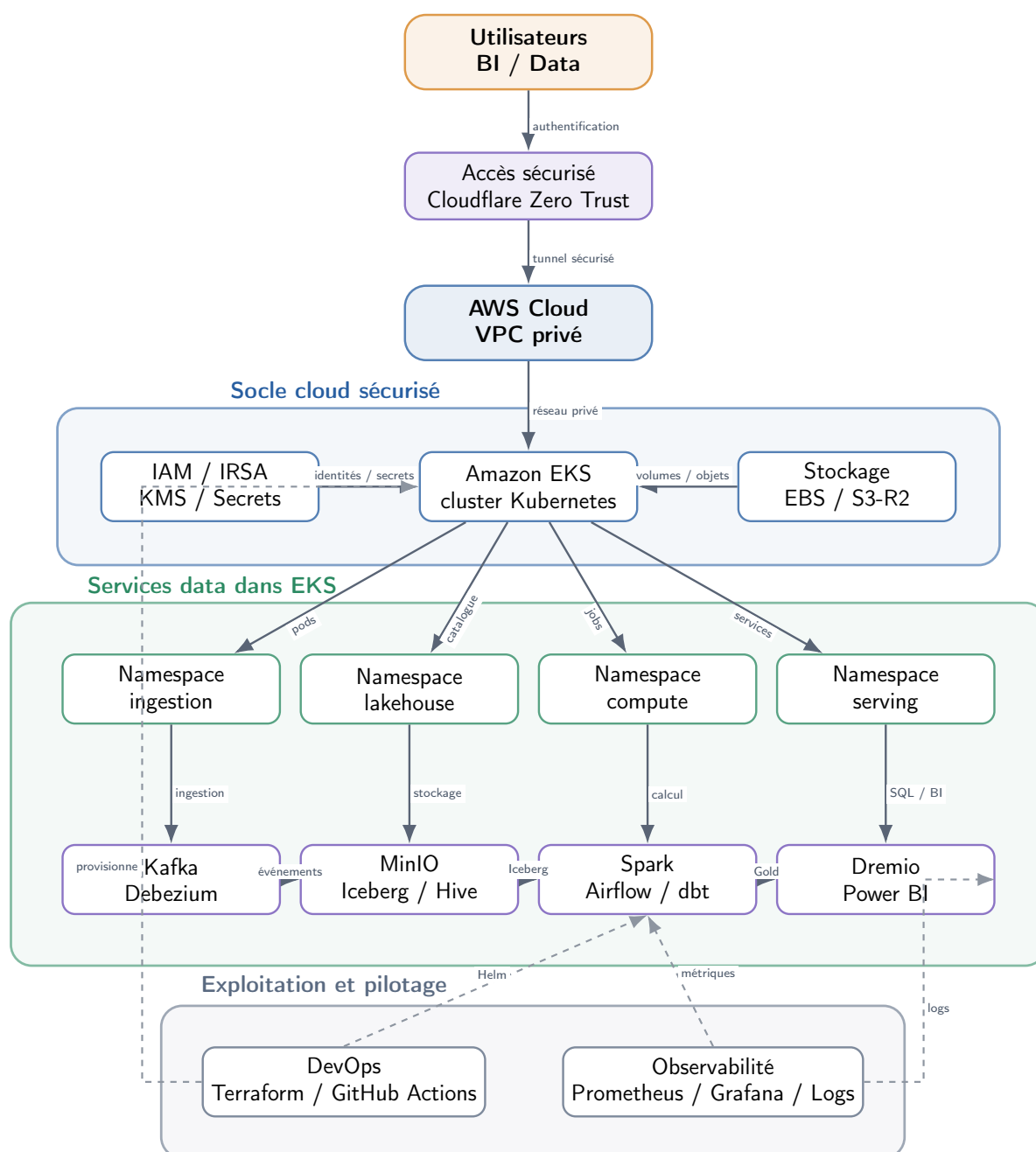


FIGURE 3.3 – Diagramme d'infrastructure cloud cible

La vue infrastructure montre la séparation entre le socle cloud, les services data et l'exploitation. Les relations annotées indiquent le sens des dépendances : Cloudflare sécurise l'accès, AWS fournit le réseau privé, IAM/IRSA/KMS portent les identités et secrets, le stockage fournit les volumes et objets, puis EKS déploie les pods dans les namespaces. Les flux data passent de Kafka vers MinIO/Iceberg, puis vers Spark/dbt et Dremio. Les flèches en pointillés décrivent les relations d'exploitation : Terraform et GitHub Actions provisionnent, tandis que Prometheus, Grafana et les logs supervisent les traitements et services.

3.6 Scénario de référence du pipeline data

Le scénario de référence sert de fil conducteur au projet. Il illustre le fonctionnement attendu d'un pipeline complet :

1. une source du périmètre alimente la plateforme : base PostgreSQL/ONE, Informix CBS, SmartStream ou fichier métier CSV/XLS/XLSX ;
2. pour une source base de données, Debezium ou un connecteur adapté capture les changements et les publie dans un topic Kafka lorsque le mode streaming est utilisé ;
3. pour une source fichier, un traitement batch valide le format et écrit les données dans la couche Bronze ;
4. un job Spark consomme le flux Kafka ou les données batch et écrit une table Iceberg Bronze ;
5. des modèles dbt transforment la donnée en tables Silver puis Gold ;
6. Dremio interroge les tables Gold et les rend disponibles pour la BI ;
7. Prometheus, Grafana, Fluent Bit et OpenObserve permettent de suivre l'exécution.

Le tableau 3.5 associe à chaque composant un critère observable afin de vérifier que le scénario de référence est effectivement couvert.

TABLE 3.5 – Critères d'acceptation du scénario de référence

Composant	Critère de validation
Sources bases / Debezium	Une modification issue de PostgreSQL/ONE, Informix CBS ou SmartStream peut être capturée ou ingérée selon le connecteur retenu.
Fichiers métiers	Un fichier CSV, XLS ou XLSX conforme est validé puis chargé vers la couche Bronze.
Kafka	Le topic attendu existe et contient les messages produits.
Spark / Iceberg	Le job Spark écrit une table Bronze visible dans le catalogue Hive.
dbt	Les modèles Silver et Gold s'exécutent sans erreur et produisent des tables requêtables.
Dremio	Les tables Gold sont accessibles par requête SQL.
Observabilité	Les métriques et logs du pipeline sont consultables dans Grafana et OpenObserve.

3.7 Contraintes de réalisation

Le projet est soumis à plusieurs contraintes :

- **contrainte de coût** : l'environnement cible est un environnement dev, dimensionné pour démontrer la solution sans reproduire tous les coûts d'une production ;

- **contrainte de secrets** : les credentials AWS, Cloudflare, R2 et GitHub ne peuvent pas être versionnés ;
- **contrainte de temps** : certaines phases sont livrées sous forme d'artefacts prêts à déployer et nécessitent une validation finale sur cluster ;
- **contrainte de périmètre** : les données réelles EQDOM et certains aspects de production, comme les sauvegardes avancées ou la haute disponibilité complète, sont hors MVP.

Conclusion du chapitre

Ce chapitre a transformé la problématique générale en exigences concrètes. Le périmètre fonctionnel couvre plusieurs familles de sources : bases opérationnelles PostgreSQL, Informix CBS et SmartStream, ainsi que fichiers métiers CSV, XLS et XLSX. Cette diversité justifie la coexistence de mécanismes d'ingestion batch et streaming, avec une zone Bronze destinée à conserver les données ingérées de manière traçable et rejouable.

Les exigences fonctionnelles décrivent ensuite la chaîne attendue de bout en bout : provisionnement du socle AWS, déploiement des namespaces Kubernetes, stockage Lakehouse, capture CDC, ingestion de fichiers, transformations Bronze–Silver–Gold, orchestration Airflow, exposition SQL avec Dremio et collecte des métriques et logs. Les exigences non fonctionnelles complètent ce périmètre en imposant des objectifs de sécurité, de maintenabilité, de reproductibilité, de scalabilité et de maîtrise des coûts.

L'identification des acteurs clarifie la gouvernance du système. Le Data Engineer construit et maintient les pipelines, l'analyste BI exploite les données Gold, l'administrateur Cloud/SRE supervise les ressources et l'encadrant technique contrôle l'alignement avec le cahier des charges. Les diagrammes de flux, d'activités et d'infrastructure relient ces responsabilités aux composants techniques et montrent comment une donnée transite depuis sa source jusqu'aux usages analytiques.

Enfin, le scénario de référence et ses critères d'acceptation fournissent une base de validation mesurable. Une source doit pouvoir être ingérée, les événements doivent être visibles dans Kafka, les traitements Spark et dbt doivent produire les tables attendues, Dremio doit rendre les données Gold requêtables et les outils d'observabilité doivent permettre le diagnostic des traitements.

L'analyse aboutit ainsi à un périmètre réaliste pour un environnement de démonstration, tout en conservant une trajectoire vers la production. Le chapitre suivant décrit l'architecture retenue pour répondre à ces exigences et organiser les composants de manière cohérente.

Chapitre 4 : CONCEPTION DE LA SOLUTION

Ce chapitre présente l'architecture cible de la Cloud Data Platform. La conception reprend la logique en sept couches décrite dans les spécifications, puis la projette sur AWS EKS, Kubernetes et les artefacts du dépôt.

4.1 Architecture globale

La plateforme comporte sept couches logiques : sources, ingestion, stockage, calcul, virtualisation, présentation et observabilité. Elle est hébergée sur **AWS EKS** [3] en environnement **dev**, avec un design compatible avec une évolution multi-cloud ou on-premise grâce à Kubernetes [6], MinIO [9] et Iceberg [10].

La figure 4.1 représente ces couches et met en évidence le chemin suivi par les données jusqu'aux usages analytiques.

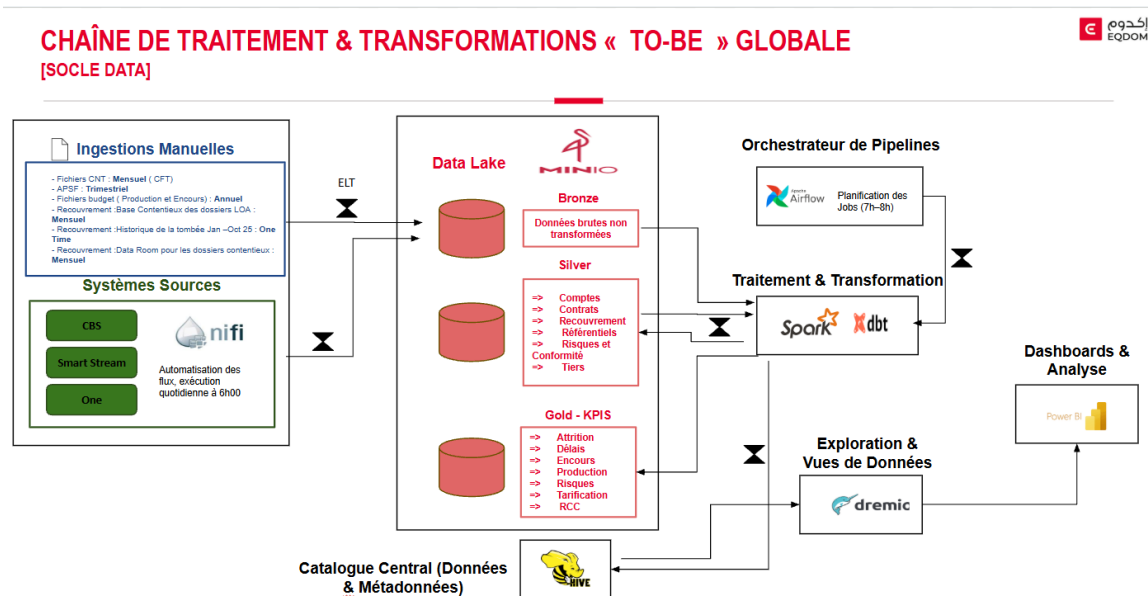


FIGURE 4.1 – Architecture logique en couches de la Cloud Data Platform

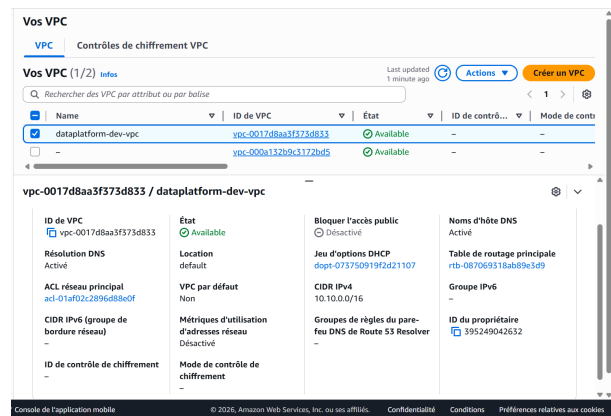
Les captures détaillées d'exécution sont présentées dans le chapitre de réalisation. Dans ce chapitre, seules les illustrations qui aident à comprendre les choix d'architecture sont conservées.

4.2 Déploiement physique sur AWS EKS

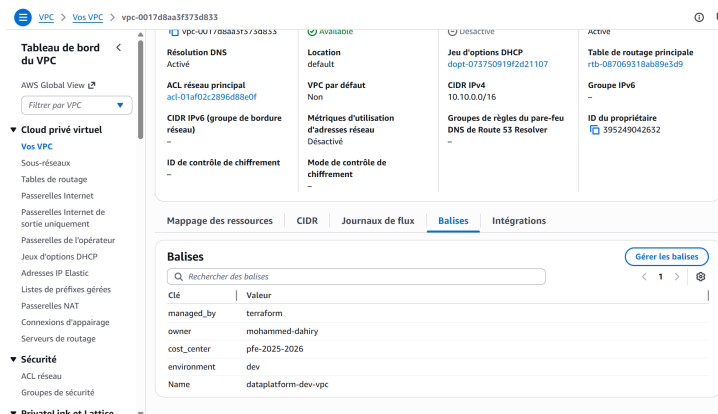
4.2.1 Landing zone

La landing zone est composée d'un VPC multi-AZ, de sous-réseaux privés et publics, d'un NAT Gateway unique en environnement `dev`, de clés KMS dédiées et d'un cluster EKS managé. Les modules Terraform `vpc`, `kms`, `iam`, `eks`, `ecr` et `lambda-scaling` isolent les responsabilités et permettent une évolution contrôlée.

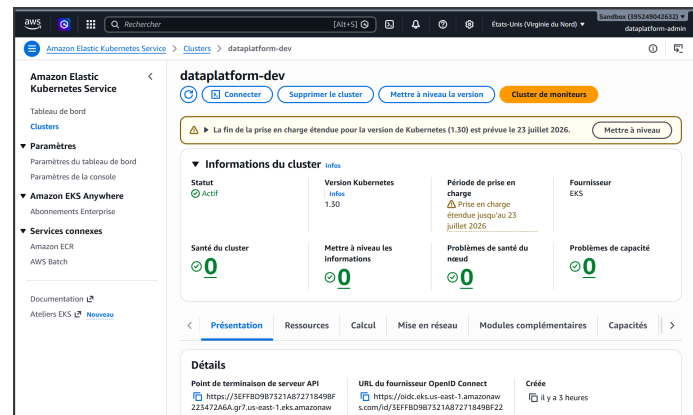
La figure 4.2 illustre les principales briques AWS de cette landing zone : réseau, routage et cluster EKS.



Vue AWS du VPC dédié à la plateforme



Organisation des sous-réseaux et des tables de routage



Vue générale du cluster EKS dataplatform-dev

FIGURE 4.2 – Socle cible AWS/EKS retenu pour la conception

4.2.2 Node groups

Deux groupes de nœuds séparent les contraintes stateful et compute :

- **stateful-ng** : workloads persistants comme MinIO, PostgreSQL, Hive et Kafka ;
- **compute-ng** : workloads élastiques comme Spark, Airflow, Dremio, observabilité et jobs ponctuels.

Cette séparation rend possible le **scale-to-zero** du compute sans arrêter les services de stockage, ce qui répond à la contrainte d'optimisation des coûts.

4.3 Stratégie des environnements DEV, UAT et PROD

La plateforme est conçue selon une progression d'environnements qui sépare les usages de développement, de validation métier et d'exploitation. Cette séparation évite de mélanger les expérimentations techniques avec les données et services destinés aux utilisateurs finaux.

La figure 4.3 présente cette progression. Le tableau 4.1 précise le rôle et le niveau d'exigence associé à chaque environnement.

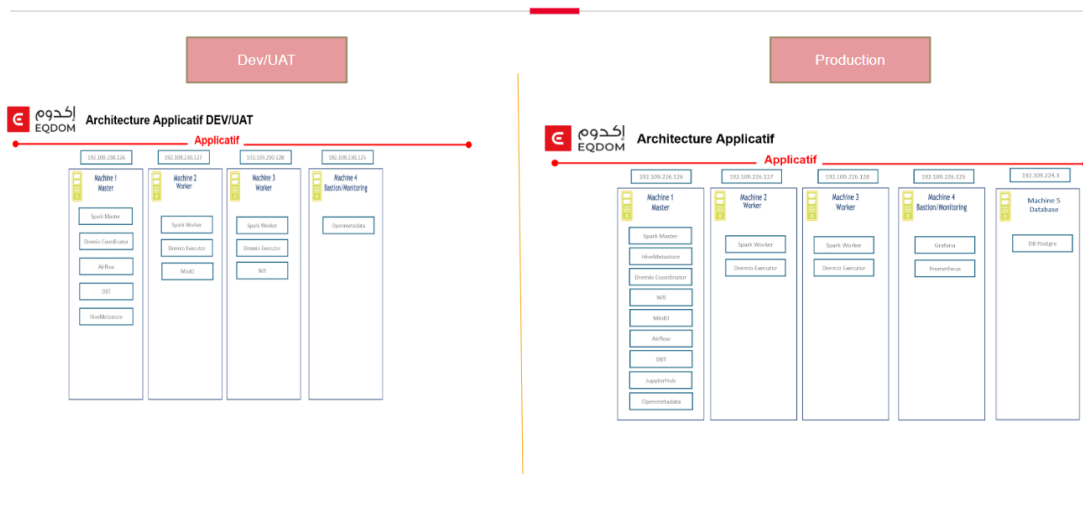


FIGURE 4.3 – Organisation cible des environnements DEV, UAT et PROD

TABLE 4.1 – Rôle des environnements de la plateforme

Environnement Rôle dans la conception	
DEV	Environnement de construction et d'expérimentation. Il sert à développer les modules Terraform, les manifests Kubernetes, les DAGs Airflow, les jobs Spark et les modèles dbt avec des ressources réduites et des données de test.
UAT	Environnement de recette. Il reproduit la chaîne fonctionnelle de bout en bout pour valider les flux, les tables Silver/Gold, les contrôles de qualité, les accès aux interfaces et les scénarios de démonstration avant promotion.
PROD	Environnement d'exploitation. Il applique des paramètres plus stricts : haute disponibilité, quotas adaptés, sauvegardes, supervision renforcée, secrets réels, contrôle d'accès plus fin et procédures de changement maîtrisées.

Dans cette logique, un changement est d'abord testé en **DEV**, stabilisé en **UAT**, puis promu vers **PROD**. Les mêmes principes d'infrastructure as code, de namespaces, de secrets et de CI/CD sont conservés entre les environnements, mais les tailles de ressources, les règles d'accès et les paramètres de résilience évoluent selon le niveau de criticité.

4.4 Stratégie des namespaces Kubernetes

Le tableau 4.2 décrit la segmentation Kubernetes retenue. Chaque namespace regroupe des workloads ayant une responsabilité technique homogène.

TABLE 4.2 – Namespaces `platform-*` alignés avec le cahier des charges

Namespace	Workloads prévus
<code>platform-ingestion</code>	Strimzi Kafka, Kafka Connect, Debezium, NiFi
<code>platform-storage</code>	MinIO Operator, MinIO Tenant, PostgreSQL source
<code>platform-metastore</code>	Hive Metastore, PostgreSQL HMS
<code>platform-compute</code>	Spark Operator, SparkApplication, dbt jobs
<code>platform-orchestr</code>	Airflow, base metadata Airflow
<code>platform-serving</code>	Dremio, services SQL/BI
<code>platform-monitoring</code>	Prometheus, Alertmanager, Grafana
<code>platform-logging</code>	Fluent Bit, OpenObserve
<code>platform-security</code>	cert-manager [31], Cloudflare Operator, secrets d'exposition

Chaque namespace reçoit une `ResourceQuota`, une `NetworkPolicy` de base et un niveau Pod Security Admission adapté. Cette stratégie limite le blast radius et facilite l'explication de l'architecture au jury.

4.5 Conception du pipeline médaillon

Le pipeline est conçu autour d'un flux principal PostgreSQL → Kafka → Iceberg, complété par une ingestion fichiers. La séparation Bronze/Silver/Gold permet d'organiser la qualité et la finalité des données.

La figure 4.4 illustre le flux cible, tandis que le tableau 4.3 explicite le rôle de chaque couche de données.

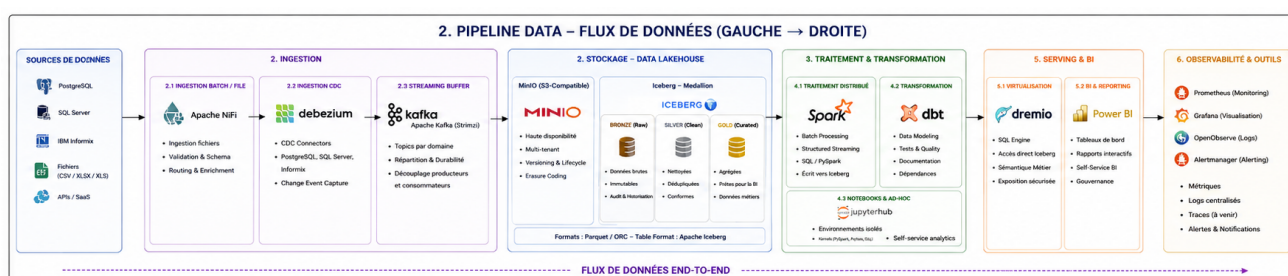


FIGURE 4.4 – Flux de données médaillon cible

TABLE 4.3 – Rôle des couches médaille

Couche	Rôle
Bronze	Données brutes, historisées, proches de la source, écrites en Iceberg pour conserver la traçabilité.
Silver	Données nettoyées, typées, dédoublées et prêtes pour les jointures métier.
Gold	Agrégats orientés analyse, indicateurs stabilisés et tables consommables par BI.

La couche d'ingestion conserve deux modes complémentaires. NiFi permet de représenter les flux semi-structurés et les scénarios d'intégration métier, tandis qu'Airflow pilote les traitements batch et dbt. Les captures concrètes de ces interfaces sont placées dans le chapitre de réalisation, car elles servent de preuve d'implémentation.

Le stockage objet est organisé par zones Bronze, Silver et Gold. Cette organisation rend visible la progression des données depuis leur état brut jusqu'aux tables analytiques.

4.6 Sécurité et gouvernance technique

La sécurité est intégrée dès la conception :

- **identité** : OIDC GitHub vers AWS et IRSA pour les controllers Kubernetes ;
- **chiffrement** : clés KMS distinctes pour EBS, secrets et logs ;
- **réseau** : API EKS privée par défaut, NetworkPolicies et exposition UI via Cloudflare Tunnel ;
- **secrets** : templates versionnés, valeurs réelles générées localement et exclues de Git ;

Les captures AWS et Cloudflare correspondantes sont déplacées dans la partie réalisation afin de montrer l'application concrète de ces principes.

4.7 Observabilité, logging et dashboarding

L'observabilité est conçue comme une couche transversale, et non comme un ajout final. Les métriques Kubernetes et applicatives sont collectées par Prometheus ; Grafana fournit les dashboards techniques ; Fluent Bit route les logs vers OpenObserve ; les ServiceMonitors ciblent les composants critiques.

Le tableau 4.4 synthétise cette conception en distinguant métriques, logs, dashboards et alertes.

TABLE 4.4 – Conception de l’observabilité

Pilier	Mise en œuvre prévue
Métriques	kube-prometheus-stack, métriques Kubernetes, Spark Operator, MinIO, CNPG, Strimzi, Airflow et Dremio.
Logs	Fluent Bit en DaemonSet, enrichissement Kubernetes, sortie OpenObserve, recherche par namespace/pod/job.
Dashboards	Grafana pour santé cluster, ressources, Kafka, PostgreSQL, Spark, Airflow et stockage ; Power BI pour indicateurs métier Gold.
Alerting	Alertmanager pour alertes techniques : pods crashloop, PVC presque pleins, jobs échoués, connecteurs CDC indisponibles.

4.8 Organisation technique des livrables

Extrait de code 4.1 – Organisation technique des livrables

```
plateforme-data/
  terraform/modules/{vpc,kms,iam,eks,ecr,lambda-scaling}
  terraform/environments/dev/
  bootstrap/phase-2..phase-10/
  docker/spark-iceberg/
  dags/
  scripts/bootstrap-phase*.sh
  docs/specs/, docs/phases/, docs/02-architecture-decision-records.md
  rapport/
```

Les captures des dossiers, registres, buckets, UIs et validations terminal sont volontairement regroupées dans le chapitre suivant, car elles décrivent la mise en œuvre réelle.

Conclusion du chapitre

Ce chapitre a traduit les exigences en une architecture cible structurée. La solution repose sur sept couches logiques : sources, ingestion, stockage, calcul, virtualisation, présentation et observabilité. Cette décomposition permet d’isoler les responsabilités, de faire évoluer les composants indépendamment et de conserver une lecture claire du parcours de la donnée.

Le socle physique retenu s’appuie sur AWS EKS et sur une landing zone décrite par Terraform. Le VPC multi-AZ, les sous-réseaux privés et publics, le NAT Gateway, les clés KMS et les rôles IAM fournissent les fondations réseau et sécurité. La séparation entre node groups stateful et compute répond à un double objectif : préserver la disponibilité des services persistants et permettre une réduction des coûts lorsque les traitements élastiques ne sont pas utilisés.

La conception prend également en compte le cycle de vie de la plateforme. Les environnements DEV, UAT et PROD distinguent construction, recette et exploitation. Les namespaces Kubernetes regroupent les workloads par fonction : ingestion, stockage, métastore, calcul, orchestration, serving, monitoring, logging et sécurité. Cette organisation facilite l’application de quotas, de politiques réseau et de règles d’accès adaptées à chaque périmètre.

Le pipeline médaille formalise ensuite le chemin des données. PostgreSQL, Kafka et les flux fichiers alimentent Bronze ; Spark et dbt assurent les transformations Silver et Gold ; Dremio expose les tables analytiques à la BI. En parallèle, Prometheus, Grafana, Fluent Bit et OpenOb-

serve constituent la couche transverse d'observabilité. La sécurité est intégrée dès la conception au travers d'OIDC, IRSA, KMS, secrets hors Git et exposition contrôlée par Cloudflare Tunnel. Enfin, l'organisation technique des livrables montre que cette conception est directement exploitable : modules Terraform, manifests Kubernetes, scripts de bootstrap, image Spark, DAGs et documentation sont structurés pour accompagner le déploiement par phases.

Le chapitre suivant confronte cette architecture à sa mise en œuvre réelle. Il présente les composants déployés, les preuves d'exécution, les validations réalisées et les éléments restant à finaliser pour renforcer l'industrialisation.

Chapitre 5 : RÉALISATION ET MISE EN ŒUVRE

Ce chapitre présente la réalisation technique du projet. Il décrit les artefacts produits, l'organisation du dépôt, l'état d'avancement par phase et les validations nécessaires pour passer du MVP technique à une plateforme pleinement exécutée sur un environnement cloud. L'objectif est de montrer que la solution ne se limite pas à une architecture théorique : elle est matérialisée par des modules Terraform, des manifests Kubernetes, des scripts d'exploitation, des DAGs et des jobs data.

5.1 Stack technologique implémentée

Le tableau 5.1 présente la stack effectivement mobilisée et relie chaque domaine technique aux outils ou artefacts correspondants.

TABLE 5.1 – Stack technologique principale

Domaine	Technologies / artefacts
Cloud	AWS VPC, EKS, EBS, ECR, KMS, Lambda, EventBridge
IaC	Terraform, backend Cloudflare R2, GitHub Actions OIDC
Kubernetes	Namespaces, RBAC, StorageClass gp3, NetworkPolicies, Helm
Stockage	MinIO Operator, MinIO Tenant, Apache Iceberg, Hive Metastore
Ingestion	Strimzi Kafka, KafkaConnect, Debezium, NiFi, Spark file ingestion
Calcul	Spark 3.5, Iceberg runtime, dbt-spark, Spark Operator
Orchestration	Airflow KubernetesExecutor, DAGs versionnés
Serving	Dremio Nautilus, source Hive, exposition SQL/BI via la couche Gold
Sécurité	cert-manager, KMS, IRSA, Cloudflare Tunnel, secrets templates
Observabilité	kube-prometheus-stack, Grafana, Fluent Bit, OpenObserve, ServiceMonitors, dashboards à finaliser
Cycle de vie	scripts <code>fasttrack</code> , <code>teardown</code> , phases de bootstrap, ArgoCD optionnel

Le tableau 5.1 ne doit pas être lu comme une simple liste d'outils. La plateforme repose sur trois ensembles complémentaires. Le premier correspond au **socle cloud et Kubernetes** : AWS

fournit le réseau, le calcul managé et les services de sécurité, tandis que Terraform décrit les ressources de manière reproductible. Kubernetes apporte ensuite un cadre commun de déploiement, d'isolation et de montée en charge pour les composants data.

Le deuxième ensemble constitue le **chemin de la donnée**. NiFi et Kafka prennent en charge les flux entrants ; MinIO, Apache Iceberg et Hive Metastore structurent le stockage Lakehouse ; Spark et dbt transforment les données ; Airflow orchestre les dépendances ; Dremio et Power BI exposent enfin les résultats aux usages analytiques. Ce découpage évite de concentrer toutes les responsabilités dans un seul moteur et facilite l'évolution progressive de chaque couche.

Le troisième ensemble regroupe les fonctions **transverses d'exploitation** : chiffrement KMS, identités IRSA, exposition Cloudflare, métriques Prometheus, dashboards Grafana et centralisation des logs. Le choix technique consiste donc à privilégier une architecture modulaire, décrite par code et exploitable par phases, plutôt qu'un assemblage monolithique difficile à maintenir.

La figure 5.1 présente les logos des principales technologies retenues, regroupées selon leur rôle dans l'architecture.



FIGURE 5.1 – Principales technologies utilisées dans la Cloud Data Platform

Rôle synthétique des principales technologies Data :

- **NiFi** représente les flux d'ingestion visuels et les intégrations de sources métier semi-structurées.
- **Kafka** fournit un transport événementiel durable et découple les producteurs des traitements consommateurs.
- **MinIO** apporte un stockage objet compatible S3 pour les zones Bronze, Silver et Gold.
- **Hive Metastore** centralise le catalogue partagé par les moteurs qui manipulent les tables analytiques.
- **Apache Iceberg** structure les tables Lakehouse avec des métadonnées, des snapshots et une évolution de schéma contrôlée.
- **Spark** exécute les traitements distribués et les écritures vers les tables Iceberg.
- **dbt** organise les transformations SQL, les modèles Silver/Gold et les contrôles associés.
- **Airflow** planifie les DAGs et orchestre les dépendances entre ingestion, transformation et publication.
- **Dremio** fournit la couche de virtualisation SQL sans imposer une duplication des données Gold.
- **Power BI** matérialise la restitution métier au travers de tableaux de bord et d'indicateurs.

Rôle synthétique des principales technologies Cloud et DevOps :

- **AWS** héberge le VPC, le cluster EKS, les volumes EBS, le registre ECR et les services de sécurité.
- **Cloudflare** sécurise l'exposition des interfaces et héberge le backend R2 utilisé pour l'état Terraform.
- **Kubernetes** fournit l'orchestration des conteneurs, l'isolation par namespaces et la gestion déclarative des workloads.
- **Terraform** décrit le socle AWS sous forme de modules versionnés et reproductibles.
- **Ansible** complète l'automatisation lorsque des séquences de configuration ou d'exploitation doivent être exécutées.
- **Docker** empaquette les dépendances nécessaires aux traitements, notamment l'image Spark Iceberg.
- **GitHub** centralise le code source, les manifests et l'historique des changements.
- **GitHub Actions** automatise les contrôles et les opérations Terraform au moyen d'une authentification OIDC.

5.2 État d'avancement par phase

Le tableau 5.2 offre une lecture synthétique de l'avancement. Il distingue les composants déjà implémentés des compléments d'industrialisation restant à finaliser.

TABLE 5.2: Matrice d'avancement et livrables par phase

Ph.	Périmètre	Artefacts disponibles	Statut
1	Landing zone AWS/EKS	Modules Terraform VPC, KMS, IAM, EKS, ECR; bootstrap namespaces, quotas, storage, réseau.	Implémenté
2	Stockage et métastore	cert-manager, MinIO, CNPG pg-source/pg-hms, Hive Metastore, secrets exemples.	Implémenté
3	Ingestion CDC	Strimzi, Kafka dev/prod, topics, KafkaConnect Debezium, NiFi.	Implémenté
4	Compute et orchestration	Docker Spark Iceberg, SparkApplications, Airflow, DAGs, dbt models.	Implémenté
5	Serving SQL/BI	Dremio Helm values, source Hive JSON, documentation Power BI.	Implémenté
6	Exposition sécurisée	Cloudflare Operator, ClusterTunnel, TunnelBindings Airflow/Dremio/Grafana/NiFi.	Implémenté
7	Observabilité	kube-prometheus-stack, OpenObserve, Fluent Bit, ServiceMonitors, bases Grafana.	À finaliser
8	CI/CD	GitHub Actions Terraform plan/apply, OIDC AWS, backend R2.	Implémenté
9	Optimisation coûts	Module Lambda scaling compute-ng, EventBridge cron.	Implémenté
10	GitOps	ArgoCD values, AppProject, Applications exemples.	À finaliser

Lecture du statut : *Implémenté* signifie que le code, les manifests, les scripts et les preuves associées sont présents dans le dépôt. À *finaliser* désigne uniquement les éléments volontairement laissés en perspective : l'import et la personnalisation des dashboards, le durcissement du monitoring/logging et l'activation GitOps complète.

Le tableau 5.2 met en évidence une progression volontairement séquencée. Les phases 1 et 2 construisent les fondations : réseau, cluster Kubernetes, sécurité, stockage objet et métastore. Les phases 3 à 5 implémentent ensuite le parcours principal de la donnée, depuis l'ingestion CDC jusqu'à l'exposition SQL/BI. Cette dépendance explique l'ordre des scripts de bootstrap : un traitement Spark ne peut pas publier de tables Iceberg tant que le stockage et le catalogue ne sont pas disponibles.

Les phases 6 à 10 concernent davantage l'exploitation et l'industrialisation. Cloudflare contrôle l'accès aux interfaces ; la couche d'observabilité prépare la surveillance des services ; GitHub Actions fiabilise les déploiements ; Lambda et EventBridge contribuent à la maîtrise des coûts ; ArgoCD prépare une évolution vers GitOps. Les statuts À *finaliser* ne remettent donc pas en cause le pipeline principal : ils identifient les compléments nécessaires pour passer d'un socle de démonstration à une exploitation durable en production.

5.3 Phase 1 — Landing zone et socle Kubernetes

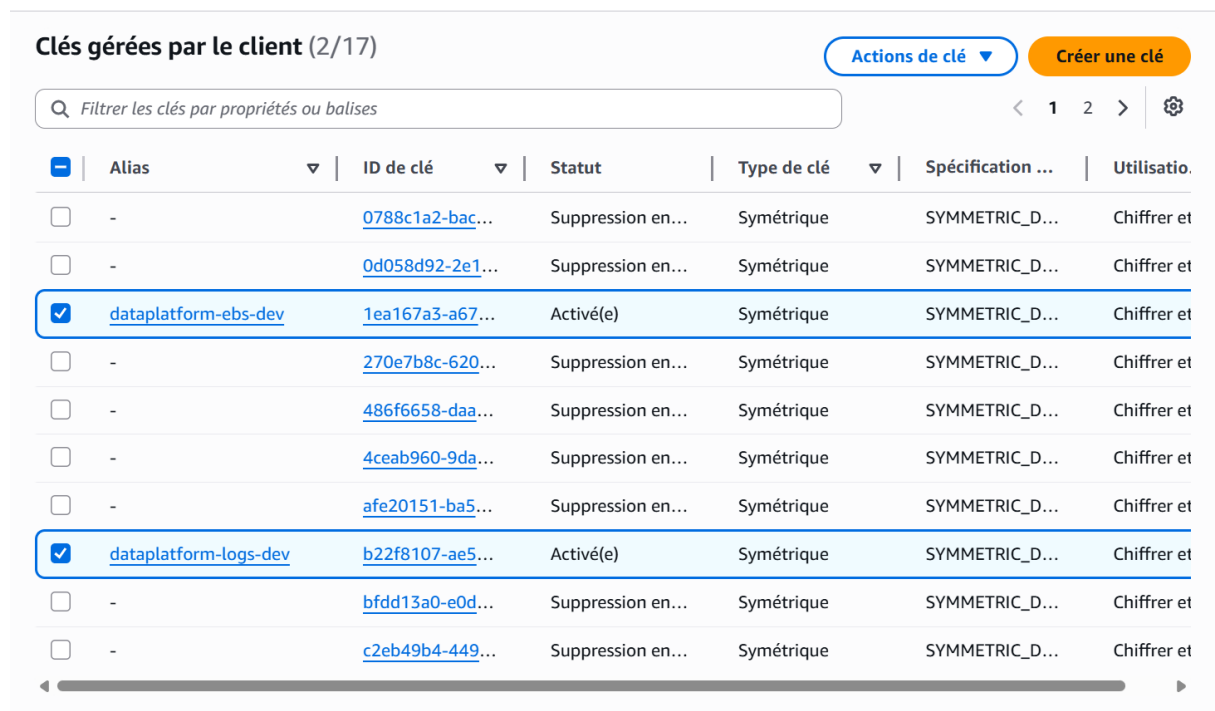
La première phase construit la fondation de la plateforme. Terraform provisionne le VPC, les sous-réseaux, le cluster EKS, les clés KMS, les rôles IAM et le registre ECR. Les choix majeurs sont documentés par les ADR : séparation des node groups, NAT unique en environnement *dev*, état Terraform sur R2 et accès EKS restreint.

Extrait de code 5.1 – Artefacts Phase 1

```
terraform/modules/{vpc,kms,iam,eks,ecr}
terraform/environments/dev/
bootstrap/{namespaces,storage-classes,resource-quotas,network-policies}
scripts/bootstrap-cluster.sh
```

Les clés KMS créées pour la plateforme matérialisent la séparation des usages de chiffrement. Elles couvrent notamment les volumes persistants, les secrets et les journaux techniques, avec une gestion côté client afin de conserver la maîtrise des clés.

La figure 5.2 montre les clés gérées côté client qui concrétisent ce choix de sécurité.



Clés gérées par le client (2/17)

Actions de clé ▼ Créer une clé

🔍 Filtrer les clés par propriétés ou balises

	Alias	ID de clé	Statut	Type de clé	Spécification ...	Utilisatio.
☐	-	0788c1a2-bac...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et
☐	-	0d058d92-2e1...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et
☒	dataplatfrom-ebs-dev	1ea167a3-a67...	Activé(e)	Symétrique	SYMMETRIC_D...	Chiffrer et
☐	-	270e7b8c-620...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et
☐	-	486f6658-daa...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et
☐	-	4ceab960-9da...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et
☐	-	afe20151-ba5...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et
☒	dataplatfrom-logs-dev	b22f8107-ae5...	Activé(e)	Symétrique	SYMMETRIC_D...	Chiffrer et
☐	-	bfd13a0-e0d...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et
☐	-	c2eb49b4-449...	Suppression en...	Symétrique	SYMMETRIC_D...	Chiffrer et

FIGURE 5.2 – Clés KMS gérées par le client pour le chiffrement de la plateforme

Le script `bootstrap-cluster.sh` applique ensuite les ressources Kubernetes transverses : namespaces, StorageClass `gp3` chiffrée, quotas et politiques réseau.

Les figures 5.3, 5.4 et 5.5 regroupent les validations du socle : contrôles terminal, workloads EKS, capacité compute, volumes persistants et configuration réseau.

```

user@DESKTOP-FUSQV7:~/projects/dataplatform-infrastructure$ kubectl -n platform-compute get pods,sparkapp
NAME                                READY   STATUS    RESTARTS   AGE
pod/spark-operator-controller-598897b9d-q95d   1/1     Running   0          25m
pod/spark-operator-webhook-5d8c3b37-d22p      1/1     Running   0          25m

NAME                                AGE
sparkapplication.sparkoperator.k8s.io/bronze-iceberg-writer
sparkapplication.sparkoperator.k8s.io/dot-run
sparkapplication.sparkoperator.k8s.io/file-to-iceberg-ingestion

user@DESKTOP-FUSQV7:~/projects/dataplatform-infrastructure$ kubectl -n platform-compute describe sparkapp
Name:         bronze-iceberg-writer
Namespace:    platform-compute
Labels:       app.kubernetes.io/part-of=dataplatfom
Annotations:  (none)
API Version:  sparkoperator.k8s.io/v1beta2
Kind:         SparkApplication
Metadata:
  Creation Timestamp: 2023-05-22T18:43:42Z
  Generation:         1
  Resource Version:    39637
  UID:                 e9e7543-8622-45ce-98f6-eb34878f272
Spec:
  Arguments:
    cdc.pg.public.agency_budget
    jmhhouse.bronze.agency_budget_rw
  Driver:
  Core Limit: 1500m
  Cores:      1
  
```

Terminal WSL de validation des namespaces et composants

```

user@DESKTOP-FUSQV7:~/projects/dataplatform-infrastructure$ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP
OS-IMAGE   KERNEL-VERSION   CONTAINER-RUNTIME
ip-10-10-52-68.ec2.internal Ready    <none>    147m  v1.30.14-eks-ecaa3a6  10.10.52.68   <none>
Amazon Linux 2 5.10.245-245.983.amzn2.x86_64 containerd://1.7.29
ip-10-10-66-51.ec2.internal Ready    <none>    147m  v1.30.14-eks-ecaa3a6  10.10.66.51   <none>
Amazon Linux 2 5.10.245-245.983.amzn2.x86_64 containerd://1.7.29
ip-10-10-88-41.ec2.internal Ready    <none>    147m  v1.30.14-eks-ecaa3a6  10.10.88.41   <none>
Amazon Linux 2 5.10.245-245.983.amzn2.x86_64 containerd://1.7.29

user@DESKTOP-FUSQV7:~/projects/dataplatform-infrastructure$ kubectl get ns
NAME                STATUS    AGE
default             Active    153m
kube-node-lease     Active    153m
kube-public          Active    153m
kube-system          Active    153m
platform-compute     Active    135m
platform-ingestion   Active    136m
platform-logging      Active    135m
platform-metastore    Active    136m
platform-monitoring   Active    135m
platform-orchestr     Active    135m
platform-security     Active    135m
platform-serving      Active    135m
platform-storage      Active    136m

user@DESKTOP-FUSQV7:~/projects/dataplatform-infrastructure$ kubectl get storageclass
NAME                PROVISIONER      RECLAIMPOLICY   VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
gp2                  kubernetes.io/aws-efs   Delete          WaitForFirstConsumer  false                  153m
gp3 (default)        ebs.csi.aws.com      Delete          WaitForFirstConsumer  true                   136m

```

Terminal WSL de validation des pods et services

```

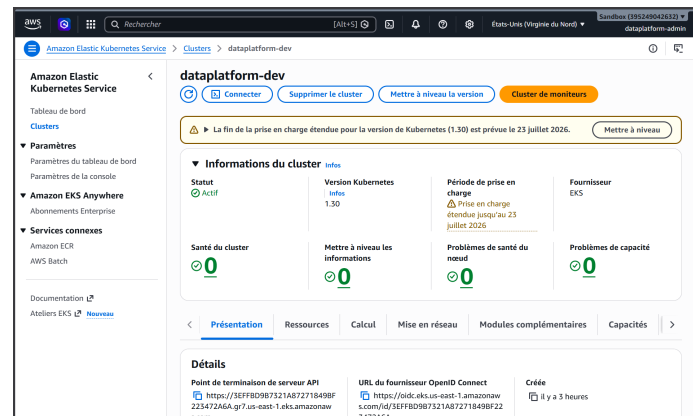
user@DESKTOP-FUSQV7:~/projects/dataplatform-infrastructure$ kubectl -n platform-storage get pods,svc
NAME                                READY   STATUS    RESTARTS   AGE
pod/cng-cloudnative-pg-59c6f957-dj75   1/1     Running   0          45m
pod/minio-operator-99d76b04-bhwd        1/1     Running   0          136m
pod/minio-operator-99d76b04-josk        1/1     Running   0          136m
pod/pg-airflow-1                         1/1     Running   0          39m
pod/pg-source-cluster-1                 1/1     Running   0          39m

NAME                                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
service/cng-webhook-service          ClusterIP    172.20.44.39     <none>         443/TCP    135m
service/operator                     ClusterIP    172.20.99.29     <none>         4221/TCP   136m
service/pg-airflow-r                 ClusterIP    172.20.123.163   <none>         5432/TCP   40m
service/pg-airflow-rw                ClusterIP    172.20.34.9      <none>         5432/TCP   40m
service/pg-airflow-rw                ClusterIP    172.20.64.142    <none>         5432/TCP   40m
service/pg-source-cluster-r          ClusterIP    172.20.120.193   <none>         5432/TCP   40m
service/pg-source-cluster-ro         ClusterIP    172.20.119.226   <none>         5432/TCP   40m
service/pg-source-cluster-rw         ClusterIP    172.20.218.45    <none>         5432/TCP   40m
service/pg-source-cluster-rw         ClusterIP    172.20.178.92    <none>         4221/TCP   136m

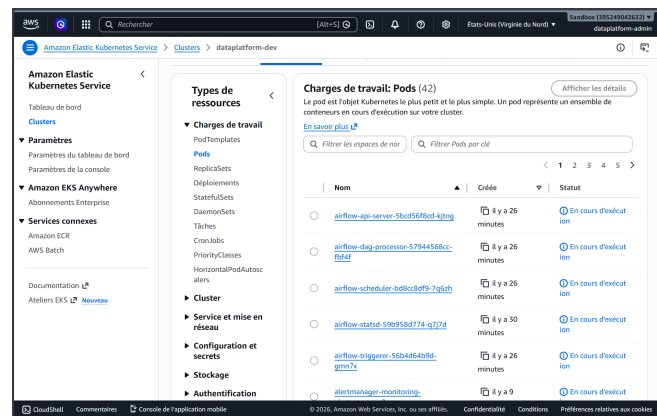
```

Terminal WSL de validation finale de l'environnement

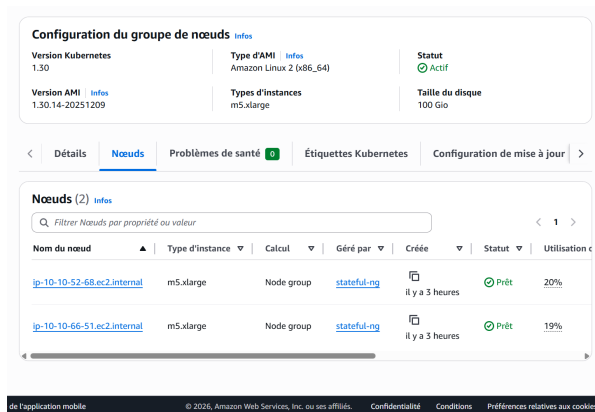
FIGURE 5.3 – Validations terminal WSL du socle déployé



Vue générale du cluster EKS



Workloads Kubernetes sur EKS



Node group compute EKS

FIGURE 5.4 – Cluster EKS, workloads et capacité compute après déploiement

Volumes (13) Informations

Last updated less than a minute ago

Ensembles de filtres enregistrés

Choisir un ensemble d... Rechercher

☐	Name	ID du volume	Type	Taille	IOPS	Débit	ID de l'inst...
☐	dataplatfom-...	vol-0a8f883cf4bc8763b	gp3	10 GiB	3000	125	-
☐	dataplatfom-...	vol-0eceb85e9a155239d	gp3	30 GiB	3000	125	-
☐	dataplatfom-...	vol-0236aacf821af7e6e	gp3	30 GiB	3000	125	-
☐		vol-05243f81fa0e837de	gp2	100 GiB	300	-	snap-0d96999
☐	dataplatfom-...	vol-0d0fcd2f1a8bd0ad3	gp3	5 GiB	3000	125	-
☐	dataplatfom-...	vol-0d3d65e0312411113	gp3	30 GiB	3000	125	-
☐		vol-04cac64c7a8b101eb	gp2	100 GiB	300	-	snap-0d96999
☐	dataplatfom-...	vol-084928786076a93fa	gp3	100 GiB	3000	125	-
☐	dataplatfom-...	vol-019070f92c3b74ded	gp3	100 GiB	3000	125	-
☐	dataplatfom-...	vol-0dac9045f365d741	gp3	100 GiB	3000	125	-
☐	dataplatfom-...	vol-058125716a8db2544	gp3	5 GiB	3000	125	-
☐		vol-038a87a1f8b1fa759	cn2	100 GiB	300	-	snan-0d96999

Volumes persistants EKS

Groupes de sécurité (1/4) Informations

Actions Exporter des groupes de sécurité au format CSV Créer un groupe de sécurité

Rechercher des groupes de sécurité par attribut ou par balise

☐	Name	ID du groupe de sécurité	Nom du groupe de sécurité	ID de VPC
☐	-	sg-0770747b0a50ec8f1	default	vpc-000a132b9c3172b
☒	dataplatfom-dev-ek...	sg-0e555c57bd6428eca	dataplatfom-dev-eks-cluster-sg	vpc-0017d8aa3f37d833
☐	-	sg-034e56e9c10c40e99	default	vpc-0017d8aa3f37d833
☐	eks-cluster-sg-datap...	sg-3c29c5faac0c5d5d2	eks-cluster-sg-dataplatfom-dev-36132...	vpc-0017d8aa3f37d833

sg-0e555c57bd6428eca - dataplatfom-dev-eks-cluster-sg

Règles entrantes Règles sortantes Partage Associations VPC Ressources connexes - nouveau

Règles sortantes (1/1)

Gérer les balises Modifier les règles sortantes

Recherche

☒	Name	ID de règle de grou...	Version IP	Type	Protocole
☒	-	sgr-0520c5b5ea92bf37d	IPv4	Tout le trafic	Tous

Security group du cluster EKS

Vos VPC

VPC Contrôles de chiffrement VPC

Vos VPC (1/2) Infos

Last updated 1 minute ago

Rechercher des VPC par attribut ou par balise

☒	Name	ID de VPC	État	ID de contr...	Mode de contr...
☒	dataplatfom-dev-vpc	vpc-0017d8aa3f37d833	Available	-	-
☐	-	vpc-000a132b9c3172bd5	Available	-	-

vpc-0017d8aa3f37d833 / dataplatfom-dev-vpc

ID de VPC vpc-0017d8aa3f37d833	État Available	Bloquer l'accès public Désactivé	Noms d'hôte DNS Activé
Résolution DNS Activé	Location default	Jeu d'options DHCP dopt-073750919f2d21107	Table de routage principale rtb-087069318ab89c349
ACL réseau principal acl-01af02c2896d88b0f	VPC par défaut Non	CIDR IPv4 10.10.0.0/16	Groupe IPv6 -
CIDR IPv6 (groupe de bordure réseau) -	Métriques d'utilisation d'adresses réseau Désactivé	Groupes de règles du pare-feu de Route 53 Resolver -	ID du propriétaire 395249042632
ID de contrôle de chiffrement -	Mode de contrôle de chiffrement -		

Vue du VPC AWS

FIGURE 5.5 – Stockage persistant, sécurité réseau et VPC AWS réalisés

Les associations entre sous-réseaux et tables de routage valident la séparation réseau du socle AWS. Les figures 5.6, 5.7 et 5.8 montrent les trois tables concernées. Les sous-réseaux publics sont reliés à la table de routage exposée via l'Internet Gateway, tandis que les sous-réseaux privés utilisent les routes sortantes contrôlées par le NAT Gateway.

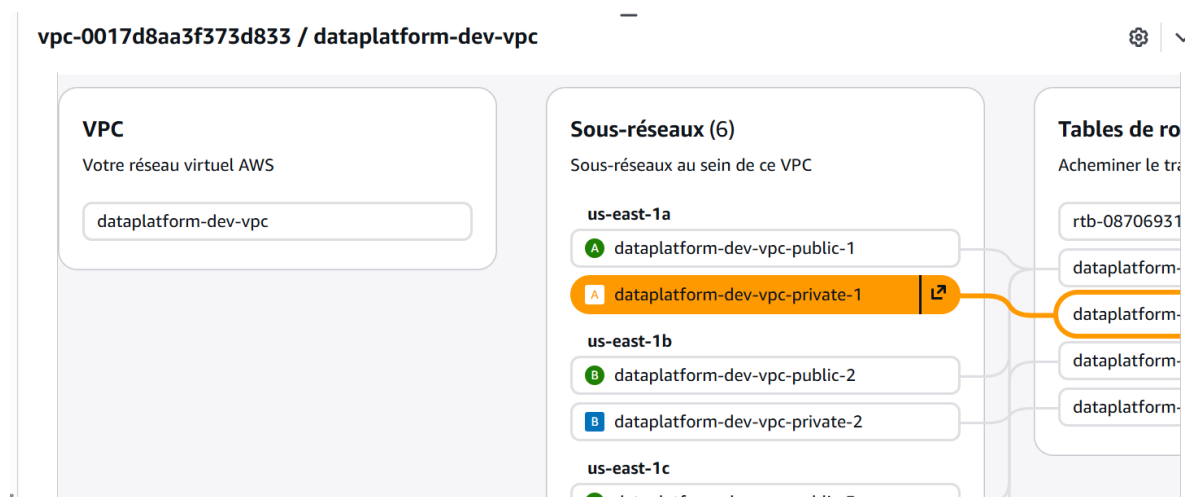


FIGURE 5.6 – Associations de la première table de routage du VPC

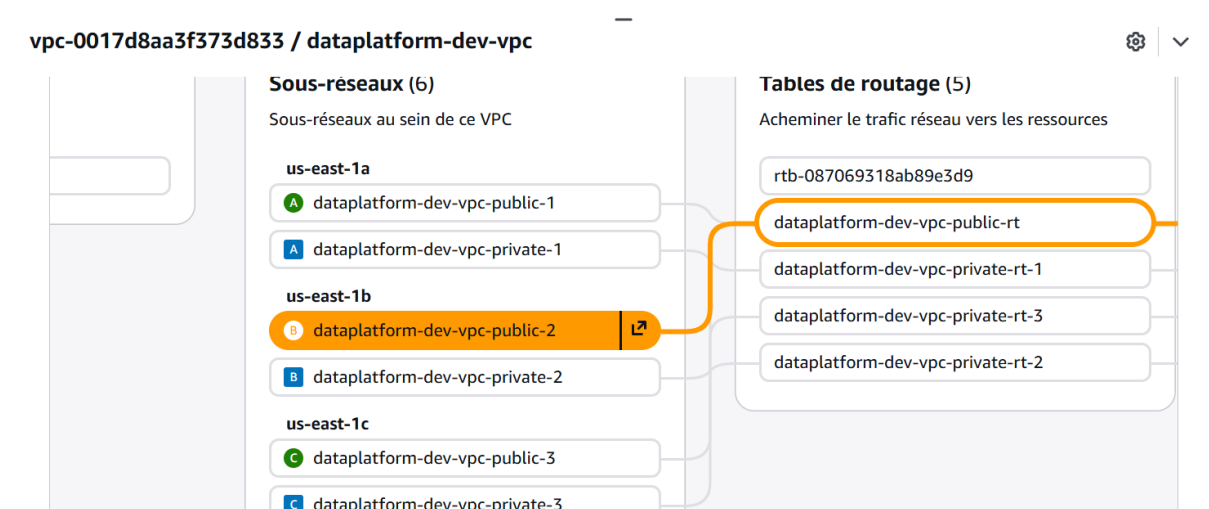


FIGURE 5.7 – Associations de la deuxième table de routage du VPC

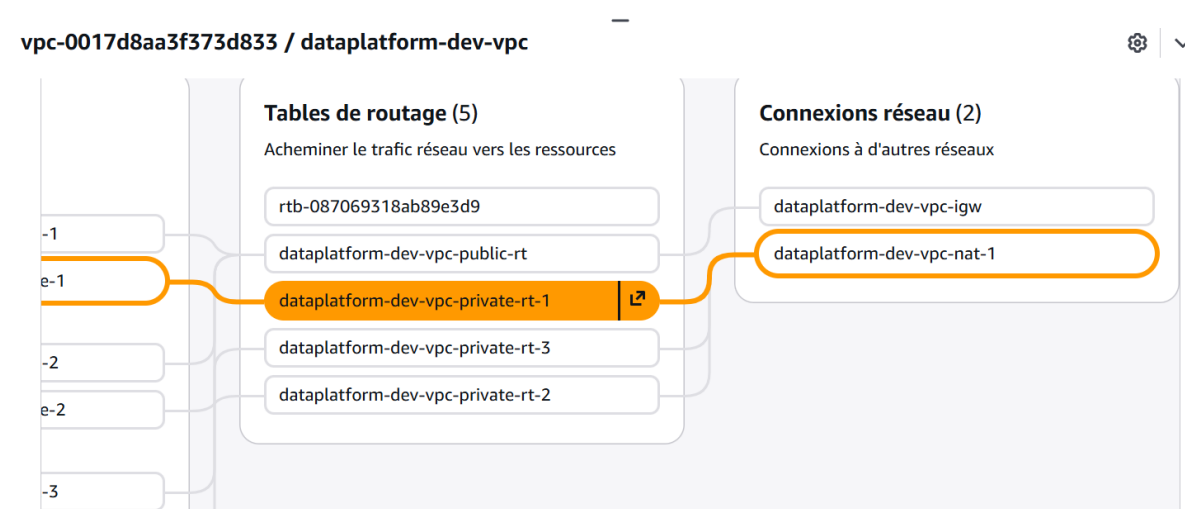


FIGURE 5.8 – Associations de la troisième table de routage du VPC

La connectivité sortante des sous-réseaux privés repose sur un NAT Gateway unique en environnement `dev`. Les figures 5.9 et 5.10 présentent respectivement cette passerelle et son adresse IP élastique. Cette association stabilise le point de sortie réseau, ce qui facilite les règles de filtrage et les diagnostics.

The screenshot displays the AWS Management Console interface for a NAT Gateway. At the top, there's a header for 'Passerelles NAT (1/1)' with an 'Infos' link and a 'Créer une passerelle NAT' button. Below this is a search bar and a filter bar showing the selected NAT Gateway ID: 'nat-0a0c57ec7e9de9f3c'. A table lists the NAT Gateway with columns: Name, ID de passerelle NAT, Type de con..., État, Message d'état, and Mod. The table shows one entry: 'dataplatfrom-dev-vp...' with ID 'nat-0a0c57ec7e9de9f3c', Type 'Public', État 'Available', and Message d'état '-'. Below the table, the breadcrumb 'nat-0a0c57ec7e9de9f3c / dataplatfrom-dev-vpc-nat-1' is shown. The 'Détails' tab is selected, showing a table with four columns: ID de passerelle NAT, Type de connectivité, État, and Message d'état. The details are as follows:

ID de passerelle NAT	Type de connectivité	État	Message d'état
nat-0a0c57ec7e9de9f3c	Public	Available	-
ARN de passerelle NAT arn:aws:ec2:us-east-1:395249042632:natgateway/nat-0a0c57ec7e9de9f3c	Adresse IPv4 publique principale 52.3.139.130	Adresse IPv4 privée principale 10.10.10.119	ID principal d'interface réseau eni-0c4928b7eac70b070
VPC vpc-0017d8aa3f373d833 / dataplatfrom-dev-vpc	Sous-réseau subnet-03c45b7ba4b66218e / dataplatfrom-dev-vpc-public-1	Créé vendredi 22 mai 2026 à 18:38:32 UTC+2	Supprimé -

FIGURE 5.9 – NAT Gateway utilisé pour la sortie contrôlée des sous-réseaux privés

Adresses IP Elastic (1/1) Informations

Trouver des adresses IP elastic par attribut ou par balise

Name	Adresse IPv4 allouée	Type	ID d'allocation
dataplatfrom-dev-vpc-nat-eip-1	52.3.139.130	Adresse IP publique	eipalloc-0b605c2172f7a0f9d

52.3.139.130

Résumé

Adresse IPv4 allouée 52.3.139.130	Type Adresse IP publique	ID d'allocation eipalloc-0b605c2172f7a0f9d	Enregistrement DNS inverse -
ID d'association eipassoc-0cf422f3916628742	Portée VPC	ID d'instance associée -	Adresse IP privée 10.10.10.119
ID d'interface réseau eni-0c4928b7eac70b070	ID du compte du propriétaire de l'interface réseau 395249042632	DNS public ec2-52-3-139-130.compute-1.amazonaws.com	ID de passerelle NAT nat-0a0c57ec7e9de9f3c (dataplatfrom-dev-vpc-nat-1)
Groupe d'adresses Amazon	Groupe de bordures réseau us-east-1	Service managed -	

FIGURE 5.10 – Adresse IP élastique associée au NAT Gateway

5.4 Phase 2 — Stockage Lakehouse et métastore

La phase 2 installe les composants persistants nécessaires au Lakehouse. Le script `bootstrap-phase2.sh` installe `cert-manager` dans `platform-security`, puis MinIO Operator, le tenant MinIO, Cloud-NativePG et Hive Metastore.

1. **MinIO** fournit le stockage objet compatible S3.
2. **CNPG** opère les bases PostgreSQL : source de démonstration et base du Hive Metastore.
3. **Hive Metastore** joue le rôle de catalogue pour les tables Iceberg.
4. **cert-manager** prépare les besoins TLS des opérateurs et services.

Cette phase matérialise le socle Bronze/Silver/Gold : les données peuvent être déposées dans des buckets ou chemins dédiés, puis référencées sous forme de tables Iceberg.

Les figures 5.11, 5.12 et 5.13 illustrent les bases PostgreSQL et les zones de stockage objet. Elles confirment la séparation entre métadonnées techniques et données organisées selon le modèle médaillon.

```

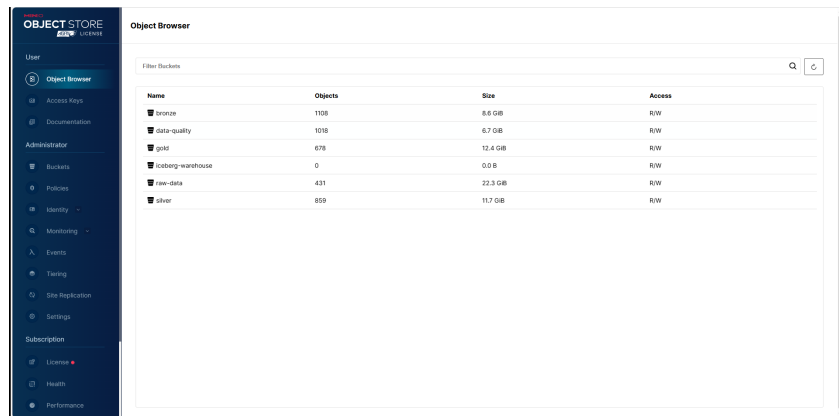
RWO      gp3      <unset>      37m
user@DESKTOP-FU5QOV7:~/projects/dataplatform-infrastructure$ kubectl -n platform-metastore get cluster,pods,pvc
NAME                                     AGE   INSTANCES   READY   STATUS                        PRIMARY
cluster.postgresql.cnpg.io/pg-hms-cluster 38m   1           1       Cluster in healthy state     pg-hms-cluster-1

NAME                                     READY   STATUS   RESTARTS   AGE
pod/hive-metastore-db-postgresql-0      1/1     Running  0          133m
pod/pg-hms-cluster-1                    1/1     Running  0          37m

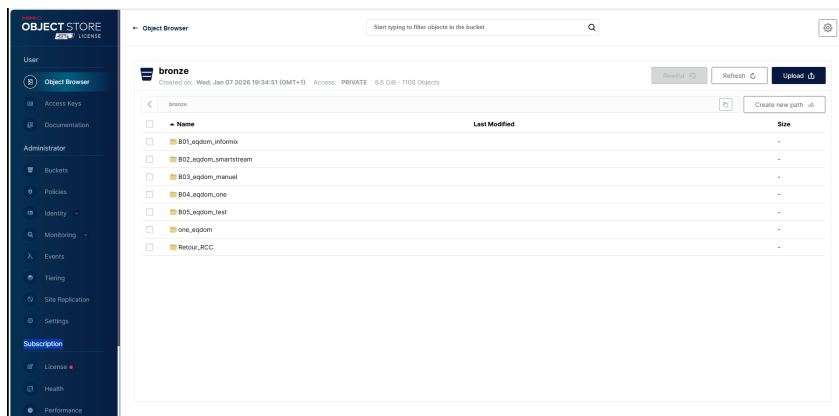
NAME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   STATUS   VOLUME
persistentvolumeclaim/data-hive-metastore-db-postgresql-0 100Gi      RWO            gp3            <unset>                  Bound    pvc-326eb1c0-6d13-45c1-bfaa-36ca9ddf6f09
persistentvolumeclaim/pg-hms-cluster-1                    100Gi      RWO            gp3            <unset>                  Bound    pvc-cd1fbf96-4493-4241-8eb4-b23589f35f56

```

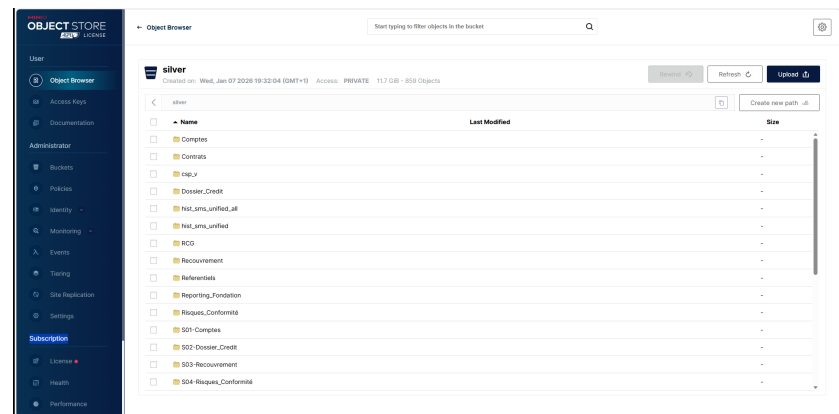
FIGURE 5.11 – Bases PostgreSQL utilisées par la source de démonstration et le Hive Metastore



Navigateur d'objets MinIO pour le Lakehouse

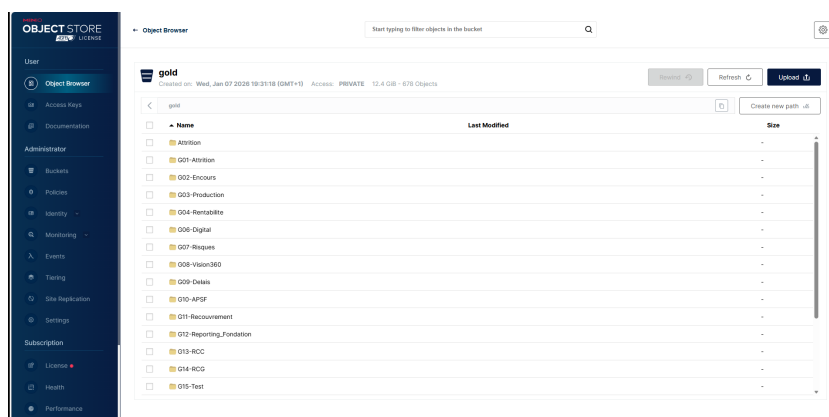


Bucket Bronze dans MinIO



Bucket Silver dans MinIO

FIGURE 5.12 – Stockage objet MinIO et premières zones Lakehouse



Bucket Gold dans MinIO

FIGURE 5.13 – Zone Gold matérialisée dans MinIO

5.5 Phase 3 — Ingestion, Kafka et CDC

La phase 3 couvre l'ingestion événementielle et les flux de données entrants. Strimzi déploie Kafka ; les topics suivent la logique médaillon ; KafkaConnect embarque Debezium pour capturer les changements PostgreSQL ; NiFi sert à l'ingestion visuelle ou semi-structurée.

Extrait de code 5.2 – Extrait conceptuel du flux CDC

```
PostgreSQL pg-source
-> Debezium KafkaConnector
-> topic cdc.pg.public.agency_budget
-> Spark bronze writer
-> Iceberg table lakehouse.bronze.cdc_pg_public_agency_budget
```

Le dépôt propose une variante **dev** légère de Kafka et une variante **prod** plus proche d'une haute disponibilité. Ce choix permet une soutenance réaliste tout en conservant une trajectoire d'industrialisation.

La figure 5.14 confirme le déploiement des composants Kafka. La figure 5.15 présente plusieurs flux NiFi orientés sources métier et complète la démonstration de l'ingestion.


```

bootstrap > phase-3 > manifests > kafka > ! kafka-connect.yaml
1 # Debezium-enabled KafkaConnect for CDC (PostgreSQL logical replication).

user@DESKTOP-FU5QOV7:~/projects/dataplatform-infrastructure$ kubectl -n platform-ingestion get kafka,kafka
topic,kafkaconnect,kafkaconnector,pods
esConnector 1

NAME                                READY   STATUS              RESTARTS   AGE
pod/nifi-0                          0/1     CrashLoopBackOff    23 (64s ago) 105m
pod/platform-connect-connect-0      1/1     Running             0           34m
pod/platform-kafka-entity-operator-55984bb88-7j7bs 2/2     Running             0           17m
pod/platform-kafka-kafka-0          1/1     Running             0           39m
pod/platform-kafka-zookeeper-0      1/1     Running             0           40m
pod/strimzi-cluster-operator-66b5ff8bbb-k5lth 1/1     Running             0           122m

user@DESKTOP-FU5QOV7:~/projects/dataplatform-infrastructure$ kubectl -n platform-storage get cluster,pods,
pvc
NAME                                AGE   INSTANCES   READY   STATUS              PRIMAR
Y
cluster.postgresql.cnpg.io/pg-airflow 37m   1           1       Cluster in healthy state pg-air
flow-1
cluster.postgresql.cnpg.io/pg-source-cluster 37m   1           1       Cluster in healthy state pg-sou
rce-cluster-1

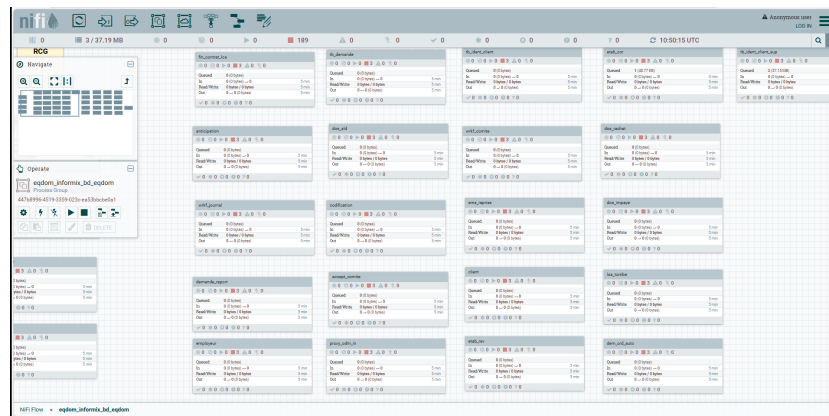
NAME                                READY   STATUS   RESTARTS   AGE
pod/cnpg-cloudnative-pg-659cf6f567-zdj75 1/1     Running  0           41m
pod/minio-operator-994d764b4-bhw2d 1/1     Running  0           133m
pod/minio-operator-994d764b4-jsb5k 1/1     Running  0           133m
pod/pg-airflow-1 1/1     Running  0           37m
pod/pg-source-cluster-1 1/1     Running  0           37m

NAME                                STATUS   VOLUME   CAPACITY
ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/pg-airflow-1 Bound        pvc-c3bad7a3-49f8-4695-8260-8d4ce85b3669 30Gi
RWO            gp3            <unset> 37m
persistentvolumeclaim/pg-source-cluster-1 Bound        pvc-bc3ef97a-da11-43da-ae9c-7cf897190508 100Gi
RWO            gp3            <unset> 37m

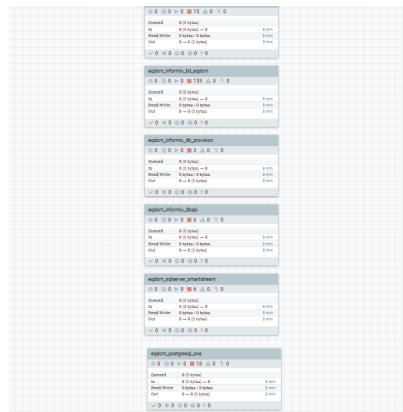
user@DESKTOP-FU5QOV7:~/projects/dataplatform-infrastructure$

```

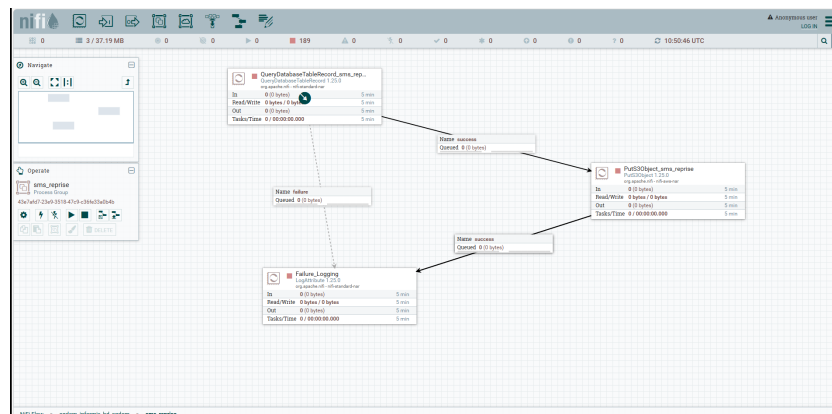
FIGURE 5.14 – Composants Kafka déployés et en fonctionnement dans Kubernetes



Flux NiFi Informix EQDOM



Flux NiFi des bases de données



Flux NiFi d'ingestion SMS reprise

FIGURE 5.15 – Flux NiFi réalisés pour l'ingestion des sources métier

5.6 Phase 4 — Spark, dbt et Airflow

La phase 4 implémente la partie centrale du pipeline data. L'image Spark Iceberg regroupe Spark, les connecteurs Iceberg/S3, les dépendances Python et le projet dbt. Les **SparkApplication** sont soumises par Airflow ou appliquées directement sur Kubernetes, ce qui permet de conserver une orchestration déclarative et reproductible.

Le tableau 5.3 liste les DAGs Airflow livrés, leur fréquence et leur responsabilité dans la chaîne de traitement.

TABLE 5.3 – DAGs Airflow livrés

DAG	Fréquence	Rôle
bronze_streaming	@once	Soumet le job Spark long-running Kafka → Bronze Iceberg.
bronze_files_daily	Quotidien	Ingestion de fichiers CSV/XLSX depuis la zone landing vers Bronze.
silver_gold_dbt	Horaire	Exécute dbt pour produire les modèles Silver et Gold.

Les modèles dbt disponibles démontrent la transformation d’une table budgétaire d’agence vers un staging Silver puis un agrégat Gold mensuel. Cette structure reste extensible à d’autres domaines métier.

Les figures 5.16, 5.17 et 5.18 montrent le calcul distribué et son orchestration. Les figures 5.19 et 5.20 présentent ensuite les résultats produits : tables Bronze, Silver et Gold, puis indicateurs métier.

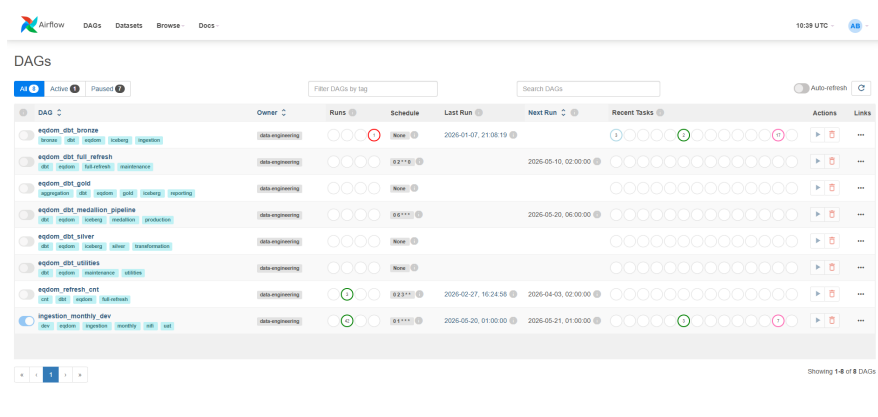
```

bootstrap > phase-3 > manifests > kafka > ! kafka-connect.yaml
1 # Debezium-enabled KafkaConnect for CDC (PostgreSQL logical replication).

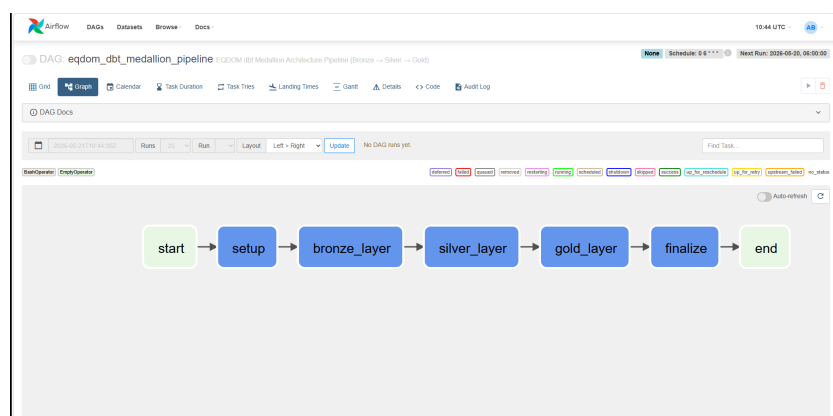
user@DESKTOP-FU5QOV7:~/projects/dataplatform-infrastructure$ kubectl -n platform-compute describe sparkapp
location bronze-iceberg-writer
Core Limit: 1500m
Cores: 1
Env Secret Key Refs:
  MINIO_ACCESS_KEY:
    Key: accessKey
    Name: minio-admin-secret
  MINIO_SECRET_KEY:
    Key: secretKey
    Name: minio-admin-secret
Memory: 2g
Node Selector:
  Workload: compute
Service Account: spark-sa
Executor:
  Cores: 1
  Env Secret Key Refs:
    MINIO_ACCESS_KEY:
      Key: accessKey
      Name: minio-admin-secret
    MINIO_SECRET_KEY:
      Key: secretKey
      Name: minio-admin-secret
  Instances: 2
  Memory: 2g
  Node Selector:
    Workload: compute
  Service Account: spark-sa
Image: 395249042632.dkr.ecr.us-east-1.amazonaws.com/dataplatform/spark-iceberg:3.5.0
Image Pull Policy: IfNotPresent
Main Application File: local:///opt/jobs/bronze_writer.py
Mode: cluster
Python Version: 3

```

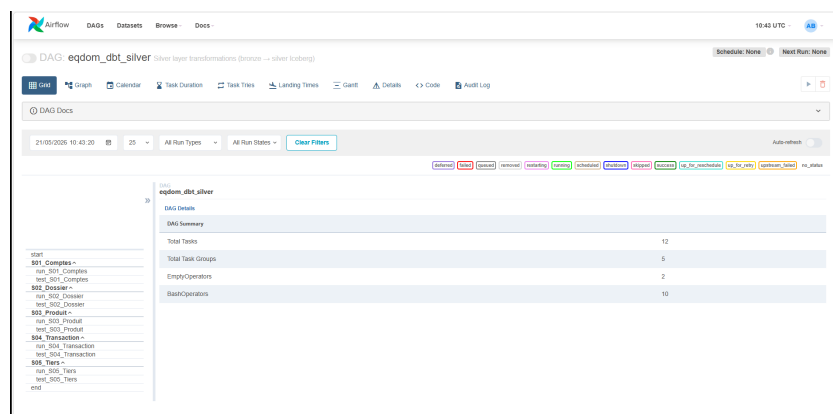
FIGURE 5.16 – Composants Spark déployés et en fonctionnement dans Kubernetes



Interface Airflow pour le suivi des DAGs



Graphe Airflow du pipeline médaillon

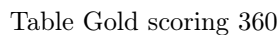
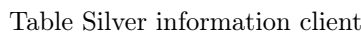


DAG dbt Silver dans Airflow

FIGURE 5.17 – Orchestration Airflow du pipeline médaillon

Interface Spark Master pour le suivi des traitements

FIGURE 5.18 – Traitements dbt, ingestion mensuelle et suivi Spark



Mohammed Dahiry — PFE Cloud Data Platform 2025–2026

The screenshot shows a Dremio SQL query interface. The query is as follows:

```
1 SELECT sha256(reference) as reference, sha256(cin) as cin, sha256(montantCredit) as montantCredit, sha256(capitalRestant) as capitalRestant, mensualite, dureeInitial, dureeRestante FROM
Gold."Vision 360"."contrats_360" where dureeRestante > 0
```

The results table has 7 columns: reference, cin, montantCredit, capitalRestant, mensualite, dureeInitial, and dureeRestante. The first few rows of data are:

reference	cin	montantCredit	capitalRestant	mensualite	dureeInitial	dureeRestante
124724420505024752489154	000720020472072472746740	37245822727246434006234882	48747206746727246434006234882	288.00	402	144
120000440071000232434347	000720020472072472746740	345444444444444444444444	48747206746727246434006234882	288.00	402	144
0000000000000000000000	000720020472072472746740	474544444444444444444444	48747206746727246434006234882	288.00	382	144
0777444444444444444444	000720020472072472746740	48747206746727246434006234882	48747206746727246434006234882	288.00	308	288

Vue des indicateurs contrats 360

FIGURE 5.20 – Indicateurs métier produits à partir de la couche Gold

5.7 Phase 5 — Serving avec Dremio

Dremio constitue la couche de virtualisation SQL. Les valeurs Helm configurent un coordinateur, un exécutateur et le stockage distribué sur MinIO. Le fichier `source-hive.json` prépare la connexion au Hive Metastore et aux propriétés `s3a`.

Dans le scénario cible, Dremio permet :

- de requêter les tables Iceberg sans déplacer les données ;
- d'exposer une interface SQL aux analystes ;
- de connecter Power BI à la couche Gold via ODBC ou Arrow Flight.

La partie BI matérialise la finalité métier de la plateforme : transformer les tables Gold en indicateurs lisibles pour le pilotage. Le cadrage distingue le contenu attendu, les objectifs de restitution et les enjeux de gouvernance afin que les tableaux de bord restent alignés avec les besoins des utilisateurs.

La figure 5.21 synthétise ce cadrage.

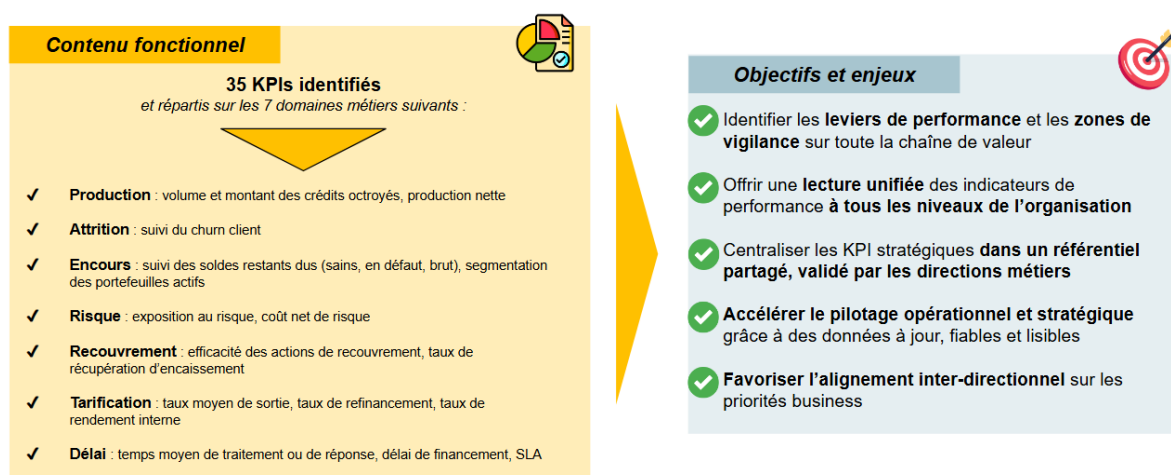


FIGURE 5.21 – Contenu, objectifs et enjeux de la couche BI

Les maquettes de dashboards illustrent ensuite la consommation des données exposées : la figure 5.22 synthétise les indicateurs de suivi et la figure 5.23 détaille les analyses métier nécessaires à l'exploration du portefeuille.

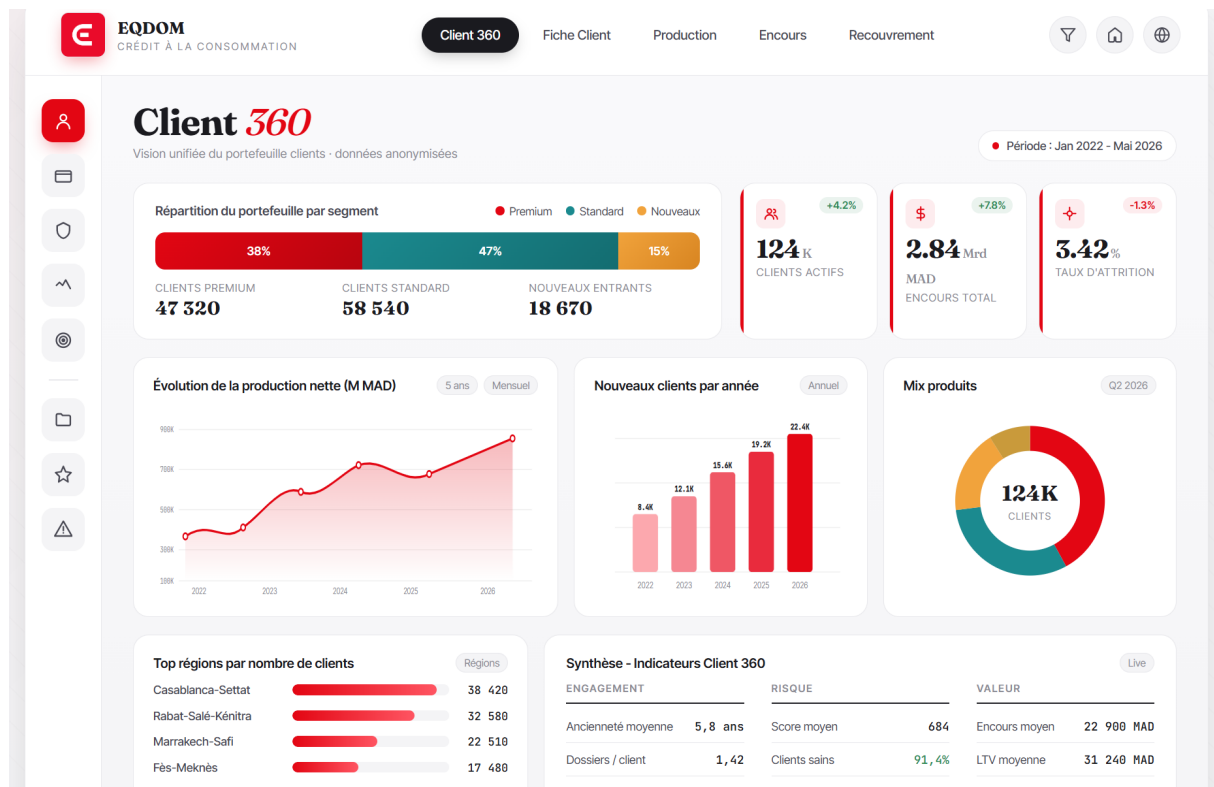


FIGURE 5.22 – Dashboard BI de synthèse alimenté par la couche Gold

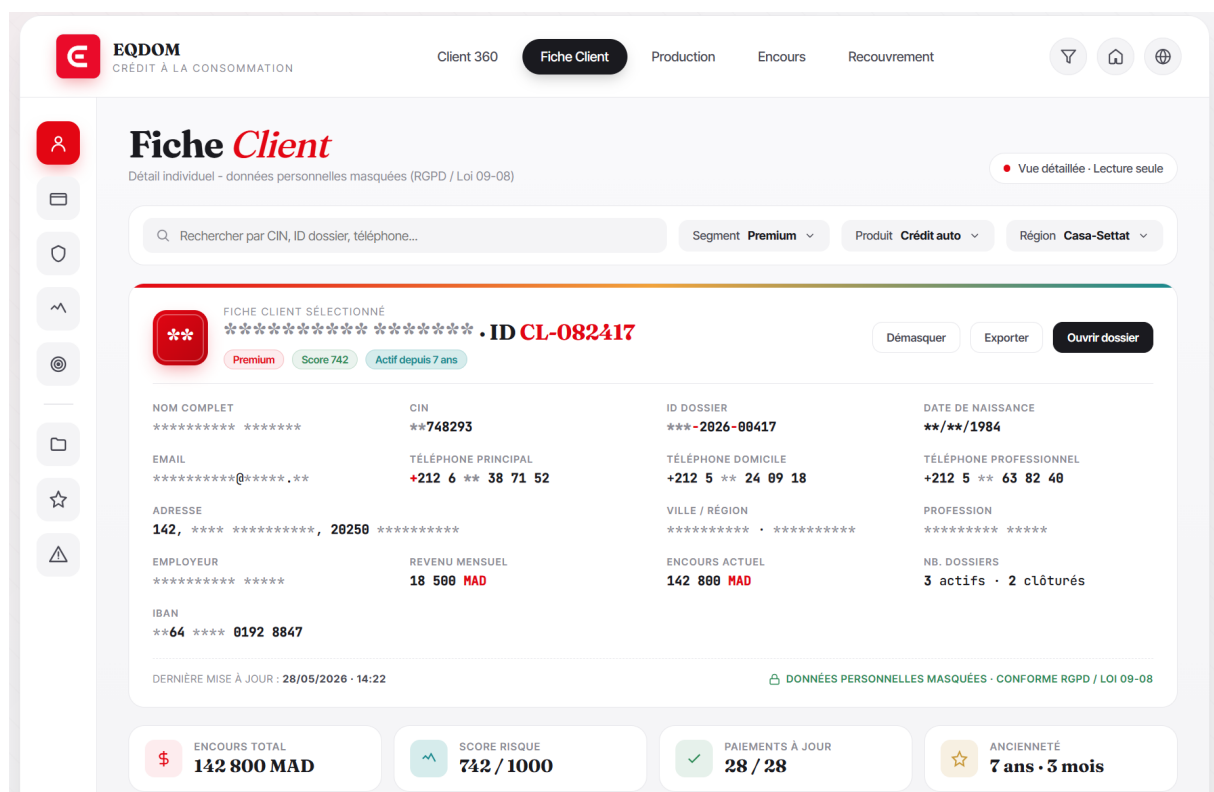


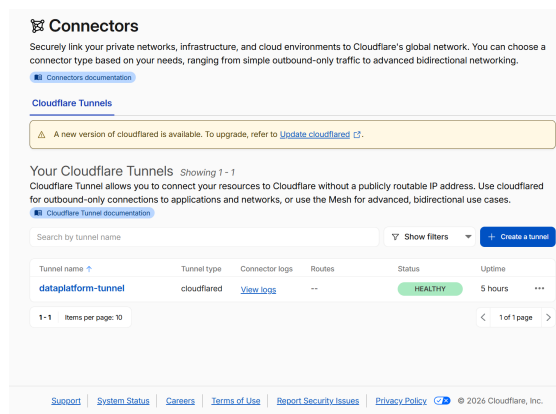
FIGURE 5.23 – Dashboard BI d'analyse métier du portefeuille

5.8 Phase 6 — Exposition sécurisée Cloudflare

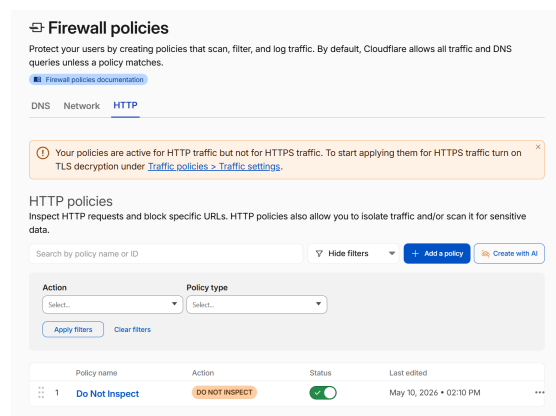
La phase 6 prépare l'accès sécurisé aux interfaces web : Airflow, Dremio, Grafana et NiFi. Le dépôt contient les manifests **ClusterTunnel** et **TunnelBinding**, mais l'activation finale dépend de secrets créés dans le dashboard Cloudflare Zero Trust.

Cette approche évite d'exposer directement des LoadBalancers publics et s'aligne avec une logique Zero Trust : authentification, politiques d'accès et publication DNS contrôlée.

La figure 5.24 présente le connecteur Tunnel et la politique de pare-feu qui matérialisent cette exposition contrôlée.



Connecteur Cloudflare Tunnel



Politique de pare-feu Cloudflare

FIGURE 5.24 – Exposition sécurisée réalisée avec Cloudflare

5.9 Phase 7 — Monitoring, logging et dashboarding

La phase 7 prépare l'observabilité de la plateforme. Le socle technique est présent dans le dépôt, mais la finalisation des dashboards, des alertes et des vues de logs reste la principale amélioration à mener après la réalisation du pipeline.

Le tableau 5.4 précise, pour chaque besoin d'exploitation, les éléments déjà disponibles et les compléments restant à finaliser.

TABLE 5.4 – Réalisation de l’observabilité

Besoin	État de réalisation
Monitoring	Socle <code>kube-prometheus-stack</code> , Prometheus, Grafana et ServiceMonitors présent ; alertes et tableaux de bord à finaliser.
Logging	Chaîne Fluent Bit vers OpenObserve structurée ; vues de logs et règles d’exploitation à compléter.
Dashboarding technique	Bases Grafana disponibles pour Kubernetes et les opérateurs ; dashboards finaux à importer ou personnaliser.
Dashboarding métier	Tables Gold et exposition BI disponibles ; participation à la mise en œuvre de dashboards métier autour d’une vision 360 des clients EQDOM et du canal digital EQDOM, c’est-à-dire les clients provenant des parcours digitaux plutôt que des agences.

Les ServiceMonitors présents ciblent notamment Spark Operator, CloudNativePG, MinIO Tenant et Strimzi. Cette partie constitue donc le principal reliquat technique du projet avec GitOps : la collecte est structurée, mais la couche d’exploitation visuelle et les alertes restent à finaliser.

5.10 Automatisation, CI/CD et cycle de vie

Le dépôt fournit plusieurs scripts pour réduire la charge opérationnelle :

- `fasttrack-infra.sh` : applique Terraform, configure `kubeconfig` et enchaîne les phases ;
- `bootstrap-phase*.sh` : déploie chaque couche dans l’ordre requis ;
- `prepare-phase*-secrets.sh` : génère des secrets locaux gitignorés ;
- `build-spark-image.sh` : construit et pousse l’image Spark vers ECR ;
- `teardown-infra.sh` : supprime Helm releases, PVC, LoadBalancers et détruit l’environnement dev.

GitHub Actions prend en charge le `terraform plan/apply` avec OIDC, ce qui évite de stocker des clés AWS longues durées dans GitHub.

La partie GitOps est conservée comme perspective d’industrialisation. Les bases ArgoCD existent, mais l’activation complète du modèle GitOps et la gouvernance des promotions DEV/UAT/PROD restent à finaliser.

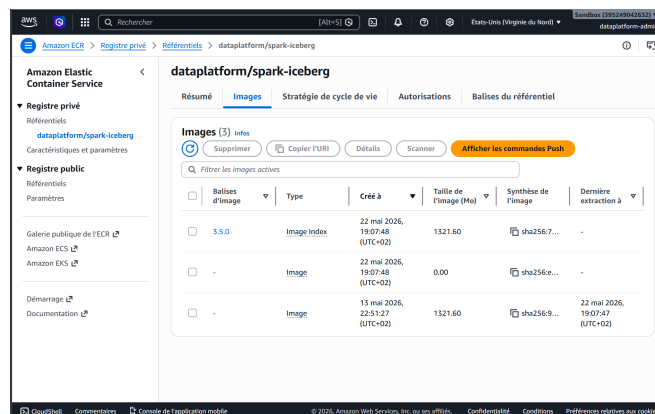
Les figures 5.25 et 5.26 complètent cette analyse par les preuves d’automatisation : organisation des dossiers, projet dbt, image Spark dans ECR, rôles IAM et backend R2.

```
[dahiry@ULE22DATAMASTER ansible]$ ll
total 828
drwxrwxr-x. 2 root root 114 24 oct. 2025 Archive CNT
drwxr-xr-x. 3 elamrani elamrani 45056 6 févr. 10:33 catalog_backup_ok
drwxrwxr-x. 6 cherkaoui cherkaoui 85 7 avril 10:26 dataplateforme-dev
drwxrwxr-x. 6 cherkaoui cherkaoui 85 7 avril 11:10 dataplateforme-dev-clean
drwxr-xr-x. 3 root root 33 22 janv. 14:52 dataplateforme-repo
drwxrwxr-x. 3 cherkaoui cherkaoui 17 6 janv. 15:12 dataplateforme-test
drwxrwxr-x. 13 root Projet 4896 30 déc. 15:23 dbt-egdom
-rw-r--r--. 1 root root 76135 10 mars 14:25 egdom_diagnostic_master_ULE22DATAMASTER_20260310_142254.txt
-rwxr-xr-x. 1 root root 34769 10 mars 14:22 egdom_diagnostic_UAT_DEV.sh
drwxrwxr-x. 7 elamrani elamrani 158 5 août 2025 hive-metastore-iceberg
drwxr-xr-x. 6 elamrani elamrani 67 27 août 2025 opt
drwxrwxr-x. 9 nafiai nafiai 174 8 mai 10:50 Projet CNT
drwxr-xr-x. 2 root root 4096 30 déc. 14:58 pyspark-scripts
drwxrwxr-x. 5 nafiai nafiai 141 25 déc. 10:08 RCC
-rw-rw-r--. 1 root root 106673 28 nov. 11:33 RCC_client_Statique.xml
-rw-rw-r--. 1 elamrani elamrani 107945 28 nov. 11:33 rcc_client.xml
-rw-rw-r--. 1 elamrani elamrani 107945 28 nov. 11:33 rcc_export.xml
-rw-rw-r--. 1 elamrani elamrani 18633 28 nov. 11:33 RCC_sample_20.xml
drwxr-xr-x. 3 root root 4096 1 sept. 2025 run
drwxrwxr-x. 2 elamrani elamrani 4096 15 août 2025 spark_custom
drwxrwxr-x. 3 elamrani elamrani 17 4 août 2025 spark_jobs
-rw-rw-r--. 1 herriadi herriadi 143331 7 avril 15:12 spark-measure_2.12-0.27.jar
-rw-rw-r--. 1 root root 145318 7 avril 10:44 spark-measure_2.13-0.27.jar
drwxrwxr-x. 9 elamrani projet 4096 20 mai 16:52 tmp
drwxr-xr-x. 6 elamrani elamrani 61 5 août 2025 var
[dahiry@ULE22DATAMASTER ansible]$
```

Organisation des dossiers d'automatisation

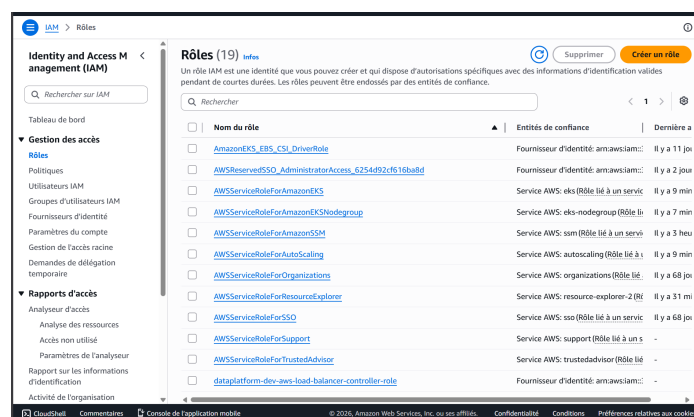
```
[dahiry@ULE22DATAMASTER dataplateforme-prod]$ cd
airflow-dags/ dbt/ .git/ nifi-flows/
[dahiry@ULE22DATAMASTER dataplateforme-prod]$ cd dbt/
[dahiry@ULE22DATAMASTER dbt]$ ll
total 272
drwxrwxr-x. 2 777 projet 22 22 janv. 15:15 analyses
drwxrwxr-x. 4 777 projet 84 5 févr. 16:46 config
drwxrwxr-x. 2 777 projet 6 22 janv. 15:15 dbt_modules
-rwxrwxr-x. 1 777 projet 3983 22 janv. 15:15 dbt_project.yml
drwxr-xr-x. 2 root projet 6 9 févr. 11:36 dbt-spark
-rw-rw-r--. 1 boudoukarra projet 748 31 mars 08:56 derby.log
drwxrwxr-x. 2 777 projet 208704 20 mai 16:41 logs
drwxrwxr-x. 8 777 projet 103 22 janv. 15:15 macros
drwxrwxr-x. 5 boudoukarra projet 133 31 mars 08:56 metastore_db
drwxrwxr-x. 6 777 projet 78 14 mai 11:34 models
-rwxrwxr-x. 1 777 projet 14 24 avril 14:13 packages.yml
drwxrwxr-x. 9 777 projet 123 16 mars 11:59 scripts
drwxrwxr-x. 2 777 projet 22 22 janv. 15:15 seeds
drwxrwxr-x. 2 777 projet 22 22 janv. 15:15 snapshots
drwxrwxr-x. 4 cherkaoui projet 128 28 avril 17:26 target
drwxrwxr-x. 2 777 projet 22 22 janv. 15:15 tests
drwxr-xr-x. 3 elghallaoui projet 21 8 avril 09:33 tmp
[dahiry@ULE22DATAMASTER dbt]$
```

Organisation du projet dbt

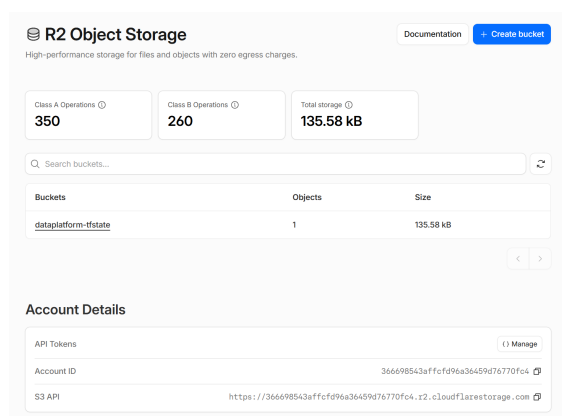


Registre ECR de l'image Spark Iceberg

FIGURE 5.25 – Artefacts d'automatisation, projet dbt et image Spark



Rôles IAM AWS



Stockage objet Cloudflare R2

FIGURE 5.26 – Identités AWS et backend R2 utilisés par l’automatisation

5.11 Difficultés rencontrées et mitigations

- **Complexité EKS** : l'ordre de déploiement a été matérialisé dans des scripts par phase.
- **Coûts AWS** : le projet introduit un teardown complet, un NAT unique en dev et un compute node group scalable à zéro.
- **Secrets** : les secrets réels ne sont pas versionnés ; seuls des exemples et scripts de génération sont fournis.
- **État Terraform** : R2 impose une gestion prudente du verrouillage ; le backend utilise `use_lockfile` et la CI limite la concurrence.

Conclusion du chapitre

Ce chapitre a présenté la matérialisation de l'architecture définie précédemment. La landing zone AWS et le socle Kubernetes ont été déployés avec Terraform et les scripts de bootstrap : VPC, sous-réseaux, tables de routage, NAT Gateway, clés KMS, rôles IAM, cluster EKS, node groups, volumes persistants et politiques réseau. Les captures et validations terminal confirment que l'infrastructure est exploitable dans l'environnement de démonstration.

La couche Lakehouse a ensuite été mise en place autour de MinIO, CloudNativePG et Hive Metastore. Les zones Bronze, Silver et Gold sont visibles dans le stockage objet, tandis que les bases PostgreSQL assurent la source de démonstration et les métadonnées du catalogue. Cette séparation constitue le socle persistant sur lequel reposent les traitements.

La chaîne d'ingestion et de transformation est également couverte. Strimzi déploie Kafka, Debezium prépare la capture des changements et NiFi illustre les flux d'intégration métier. Spark, Iceberg, dbt et Airflow prennent en charge le calcul distribué, l'organisation des modèles et l'orchestration des DAGs. Les résultats observés dans les tables Bronze, Silver et Gold ainsi que dans les indicateurs métier montrent que le pipeline dépasse le stade de la conception théorique. La restitution complète ce parcours. Dremio fournit la couche SQL destinée aux consommateurs analytiques, les dashboards BI illustrent les usages métier et Cloudflare Tunnel permet une exposition contrôlée des interfaces web. L'automatisation s'appuie sur des scripts de cycle de vie, GitHub Actions avec OIDC, un backend Terraform distant sur R2 et une image Spark publiée dans ECR.

La réalisation couvre donc le périmètre principal du projet : infrastructure, stockage, ingestion, orchestration, transformations, serving, sécurité et automatisation. Les limites restantes sont identifiées et circonscrites : enrichissement des dashboards Grafana, finalisation des alertes et des vues OpenObserve, puis activation complète de GitOps avec ArgoCD pour gouverner les promotions entre DEV, UAT et PROD.

Le chapitre suivant dresse le bilan global du Projet de Fin d'Études, présente ces limites avec recul et propose les perspectives d'évolution vers une plateforme davantage industrialisée.

Chapitre 6 : CONCLUSION ET PERSPECTIVES

6.1 Bilan du projet

Ce Projet de Fin d'Études, conduit chez **SEVENAPP** en mission **consultant Data** chez **EQ-DOM**, a permis de concevoir et de réaliser une **Cloud Data Platform** alignée sur un cahier des charges académique exigeant : architecture Lakehouse médaillon, Kubernetes operators, Infrastructure as Code, sécurité, automatisation et optimisation des coûts.

Les apports principaux :

- une **landing zone AWS/EKS** modulaire (Terraform, IRSA, KMS, deux node groups) ;
- une **chaîne data bout en bout** réalisée autour de l'ingestion, du stockage Iceberg, des transformations Spark/dbt et du serving avec Dremio ;
- une **industrialisation** par scripts, CI/CD, backend Terraform distant et ADR ;
- une **base d'observabilité** prête à être finalisée pour le monitoring, le logging et les dashboards.

6.2 Limites

Le périmètre principal du projet est réalisé : l'infrastructure cloud, le socle Kubernetes, les composants Lakehouse, l'ingestion, l'orchestration, les traitements, le serving SQL/BI et l'automatisation sont en place. Les limites restantes sont volontairement circonscrites :

- la finalisation du dashboarding, du monitoring et du logging : import des dashboards Grafana, règles d'alerting, vues OpenObserve et tableaux de bord métier ;
- l'activation complète de GitOps avec ArgoCD pour piloter les promotions DEV/UAT/-PROD de manière totalement déclarative.

6.3 Perspectives

- finaliser les dashboards techniques et métier à partir des métriques Kubernetes, Spark, Kafka, MinIO, Airflow et des tables Gold ;
- compléter les règles d'alerting, les vues OpenObserve et les procédures d'exploitation des logs ;
- activer GitOps avec ArgoCD, un dépôt d'applications dédié et des règles de promotion entre DEV, UAT et PROD ;
- enrichir progressivement la qualité de données et les alertes fonctionnelles sur la fraîcheur, la complétude et la cohérence des données.

6.4 Apports personnels

Ce stage a consolidé les compétences en **data engineering cloud-native** : Terraform, Kubernetes operators, streaming CDC et architecture médaillon. Il a également permis de participer à la mise en œuvre de dashboards métier liés à une vision 360 des clients d'**EQDOM**, ainsi qu'à l'analyse du canal digital d'**EQDOM**, en distinguant les clients issus des parcours digitaux de ceux provenant du réseau d'agences. Ces acquis sont directement transférables aux projets de transformation data du secteur financier.

Mohammed Dahiry

Bibliographie

- [1] *SEVEN – Votre Digital & Data Business Partner*. URL : <https://seven-app.org/> (visit  le 24/05/2026).
- [2] *EQDOM – Cr dit rapide en ligne et financement*. URL : <https://www.eqdom.ma/> (visit  le 24/05/2026).
- [3] *Amazon Elastic Kubernetes Service Documentation*. Amazon Web Services. URL : <https://docs.aws.amazon.com/eks/> (visit  le 24/05/2026).
- [4] Michael ARMBRUST et al. *Lakehouse : A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics*. URL : https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf (visit  le 01/06/2026).
- [5] *What is a Medallion Architecture ?* Databricks. URL : <https://www.databricks.com/glossary/medallion-architecture> (visit  le 01/06/2026).
- [6] *Kubernetes Documentation*. Cloud Native Computing Foundation. URL : <https://kubernetes.io/docs/> (visit  le 24/05/2026).
- [7] *Terraform Documentation*. HashiCorp. URL : <https://developer.hashicorp.com/terraform/docs> (visit  le 24/05/2026).
- [8] *Cloudflare R2 Documentation*. Cloudflare. URL : <https://developers.cloudflare.com/r2/> (visit  le 01/06/2026).
- [9] *MinIO Documentation*. MinIO. URL : <https://min.io/docs/minio/kubernetes/upstream/> (visit  le 24/05/2026).
- [10] *Apache Iceberg Documentation*. Apache Software Foundation. URL : <https://iceberg.apache.org/docs/latest/> (visit  le 24/05/2026).
- [11] *Apache Hive Documentation*. Apache Software Foundation. URL : <https://hive.apache.org/> (visit  le 01/06/2026).
- [12] *Apache Kafka Documentation*. Apache Software Foundation. URL : <https://kafka.apache.org/documentation/> (visit  le 01/06/2026).
- [13] *Strimzi Documentation*. Strimzi. URL : <https://strimzi.io/docs/> (visit  le 24/05/2026).
- [14] *Debezium Documentation*. Debezium. URL : <https://debezium.io/documentation/> (visit  le 24/05/2026).
- [15] *Apache Spark Documentation*. Apache Software Foundation. URL : <https://spark.apache.org/docs/latest/> (visit  le 24/05/2026).
- [16] *dbt Documentation*. dbt Labs. URL : <https://docs.getdbt.com/> (visit  le 24/05/2026).
- [17] *Apache Airflow Documentation*. Apache Software Foundation. URL : <https://airflow.apache.org/docs/> (visit  le 24/05/2026).

- [18] *Dremio Documentation*. Dremio. URL : <https://docs.dremio.com/> (visit  le 24/05/2026).
- [19] *Prometheus Documentation*. Prometheus. URL : <https://prometheus.io/docs/> (visit  le 24/05/2026).
- [20] *Grafana Documentation*. Grafana Labs. URL : <https://grafana.com/docs/grafana/latest/> (visit  le 24/05/2026).
- [21] *Fluent Bit Documentation*. Fluent Bit. URL : <https://docs.fluentbit.io/manual> (visit  le 01/06/2026).
- [22] *OpenObserve Documentation*. OpenObserve. URL : <https://openobserve.ai/docs/> (visit  le 24/05/2026).
- [23] *Helm Documentation*. Helm. URL : <https://helm.sh/docs/> (visit  le 01/06/2026).
- [24] *CloudNativePG Documentation*. CloudNativePG. URL : <https://cloudnative-pg.io/documentation/> (visit  le 01/06/2026).
- [25] *Cloudflare Zero Trust Documentation*. Cloudflare. URL : <https://developers.cloudflare.com/cloudflare-one/> (visit  le 24/05/2026).
- [26] *Apache NiFi Documentation*. Apache Software Foundation. URL : <https://nifi.apache.org/documentation/> (visit  le 01/06/2026).
- [27] *Power BI Documentation*. Microsoft. URL : <https://learn.microsoft.com/power-bi/> (visit  le 01/06/2026).
- [28] *Operator Pattern*. Kubernetes. URL : <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/> (visit  le 01/06/2026).
- [29] *GitHub Actions Documentation*. GitHub. URL : <https://docs.github.com/en/actions> (visit  le 01/06/2026).
- [30] *Argo CD Documentation*. Argo Project. URL : <https://argo-cd.readthedocs.io/> (visit  le 01/06/2026).
- [31] *cert-manager Documentation*. cert-manager. URL : <https://cert-manager.io/docs/> (visit  le 01/06/2026).